CHƯƠNG V

CÂY (TREE)

Nội dung chính

1. Tổng quan về cấu trúc cây
2. Cây nhị phân
3. Cây nhị phân tìm kiếm
4. Cây nhị phân tìm kiếm cân bằng

1. Tổng quan về cấu trúc cây

- **Định nghĩa:**

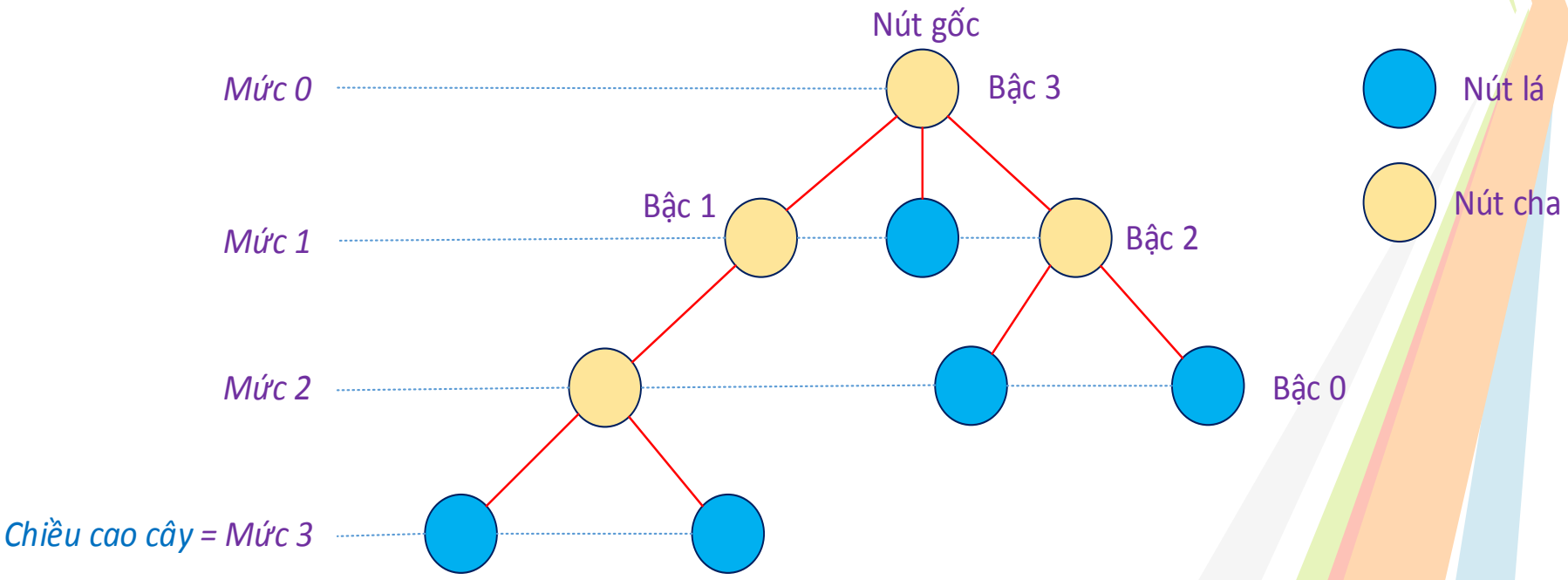
Cây là một tập hợp T các phần tử (gọi là nút của cây), trong đó có một nút đặc biệt gọi là nút gốc, các nút còn lại được chia thành những tập rời nhau T_1, T_2, …, T_n theo quan hệ phân cấp, trong đó T_i cũng là 1 cây.

Mỗi nút ở cấp i sẽ quản lý một số nút ở cấp i+1. Quan hệ này người ta gọi là quan hệ cha – con.

Một số khái niệm cơ bản

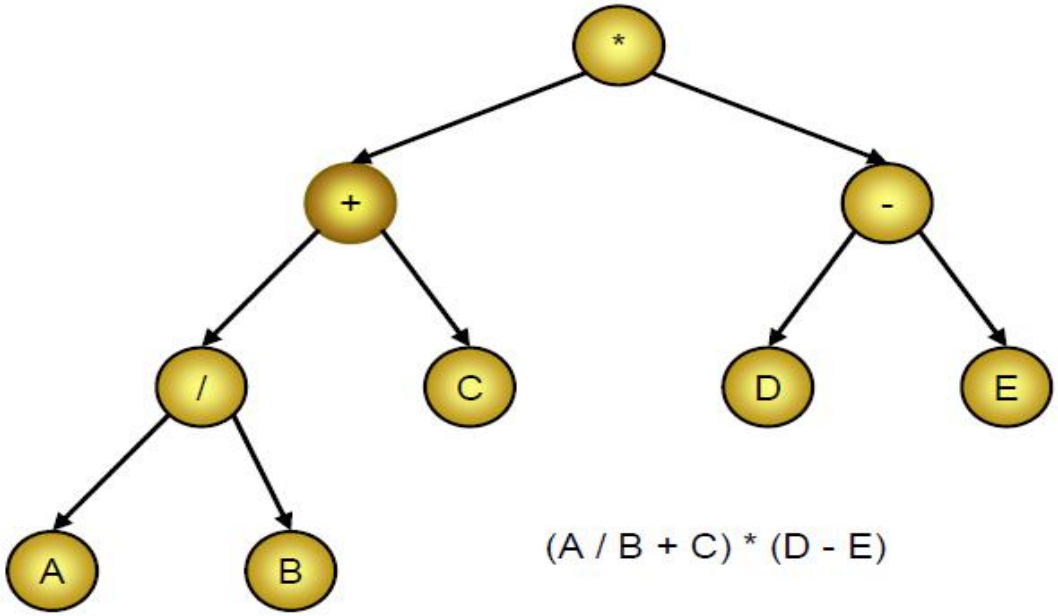
- ***Bậc của một nút***: *Là số cây con của nút đó.*
- ***Bậc của một cây***: *Là bậc lớn nhất của các nút trong cây.*
- ***Nút gốc (root)***: *Là nút không có nút cha.*
- ***Nút lá (leaf)***: *Là nút có bậc bằng 0.*
- ***Nút trung gian (interior node):*** nút có bậc khác 0 và không phải là nút gốc.
- ***Mức của một nút***:
  - *Mức của nút gốc bằng 0.*
  - *Mức của các nút con bằng mức của nút cha cộng thêm 1.*
- ***Độ dài đường đi từ gốc đến nút x***: *Là số nhánh cần đi qua kể từ gốc đến x.*
- ***Chiều cao của cây***: *Là mức lớn nhất trong cây.*

Một số khái niệm cơ bản



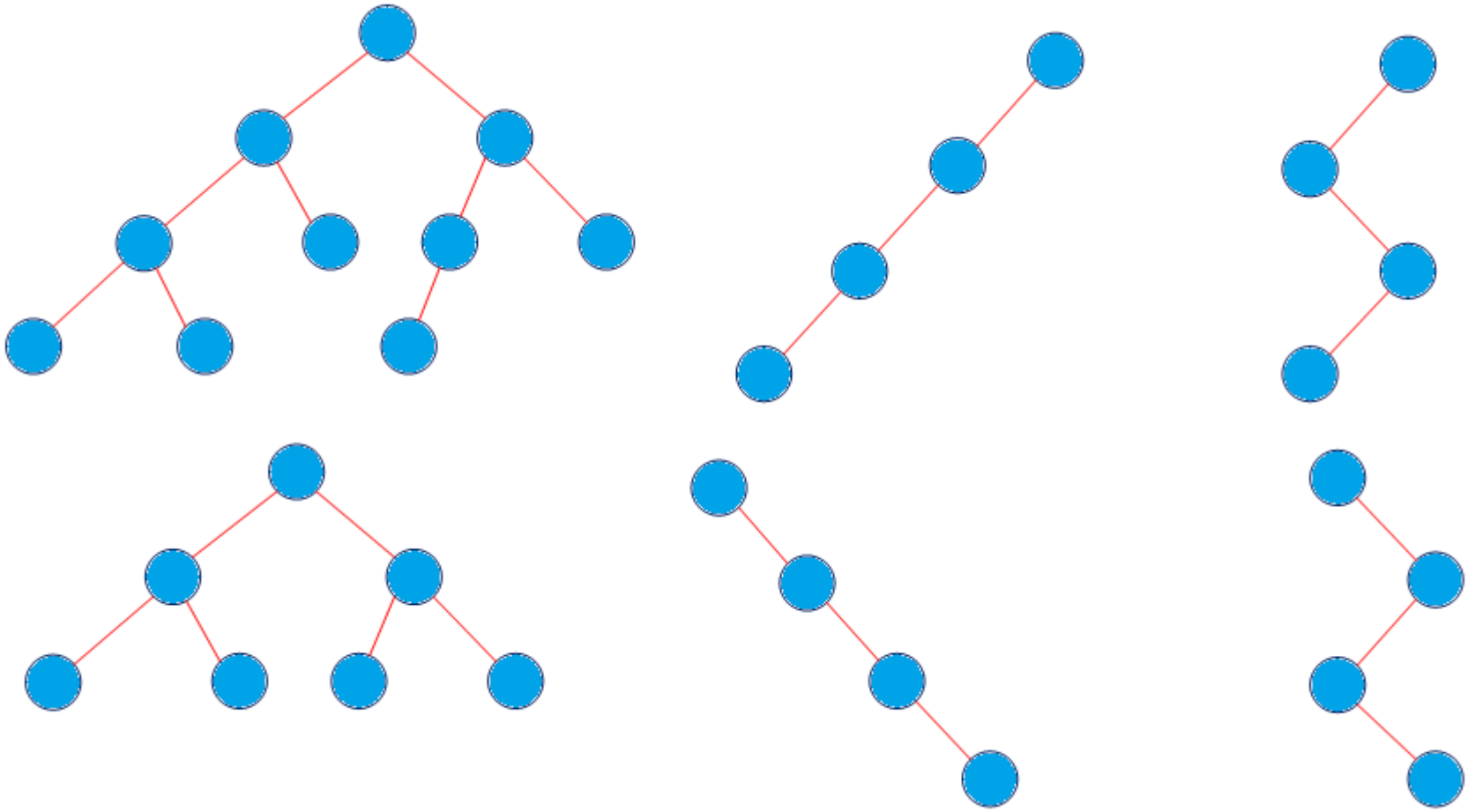
Một số ví dụ

- Sơ đồ tổ chức của 1 công ty.
- Mục lục của một cuốn sách.
- Gia phả của một họ tộc.
- Cấu trúc của thư mục trên ổ đĩa.
- Biểu thức toán học cũng có thể biểu diễn bằng cây.



2. Cây nhị phân (Binary tree)

Khái niệm: là cây có đặc điểm là mọi nút trên cây chỉ có tối đa hai nhánh con.



2. Cây nhị phân (Binary tree)

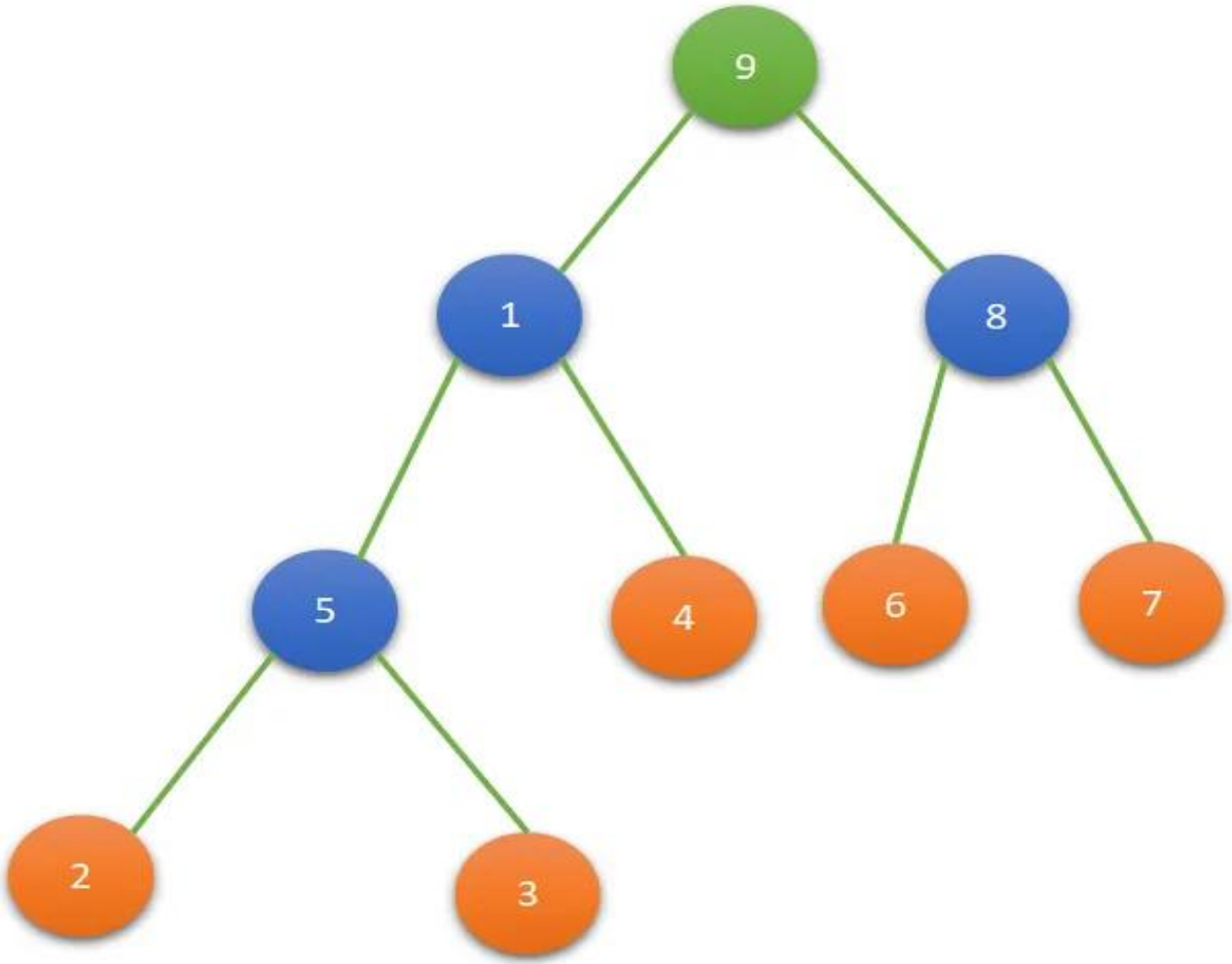


Figure: Full Binary Tree

2. Cây nhị phân (Binary tree)

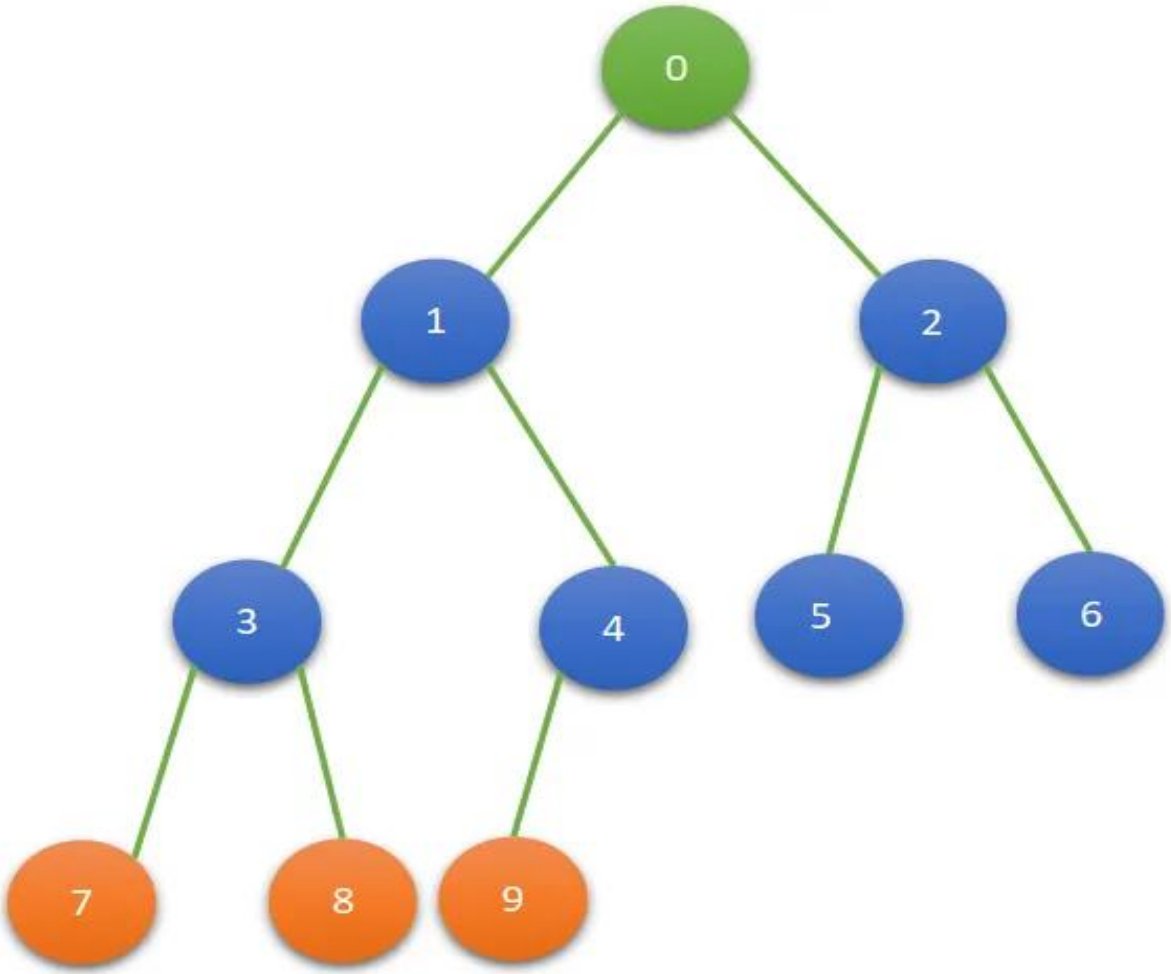


Figure: Complete Binary Tree

2. Cây nhị phân (Binary tree)

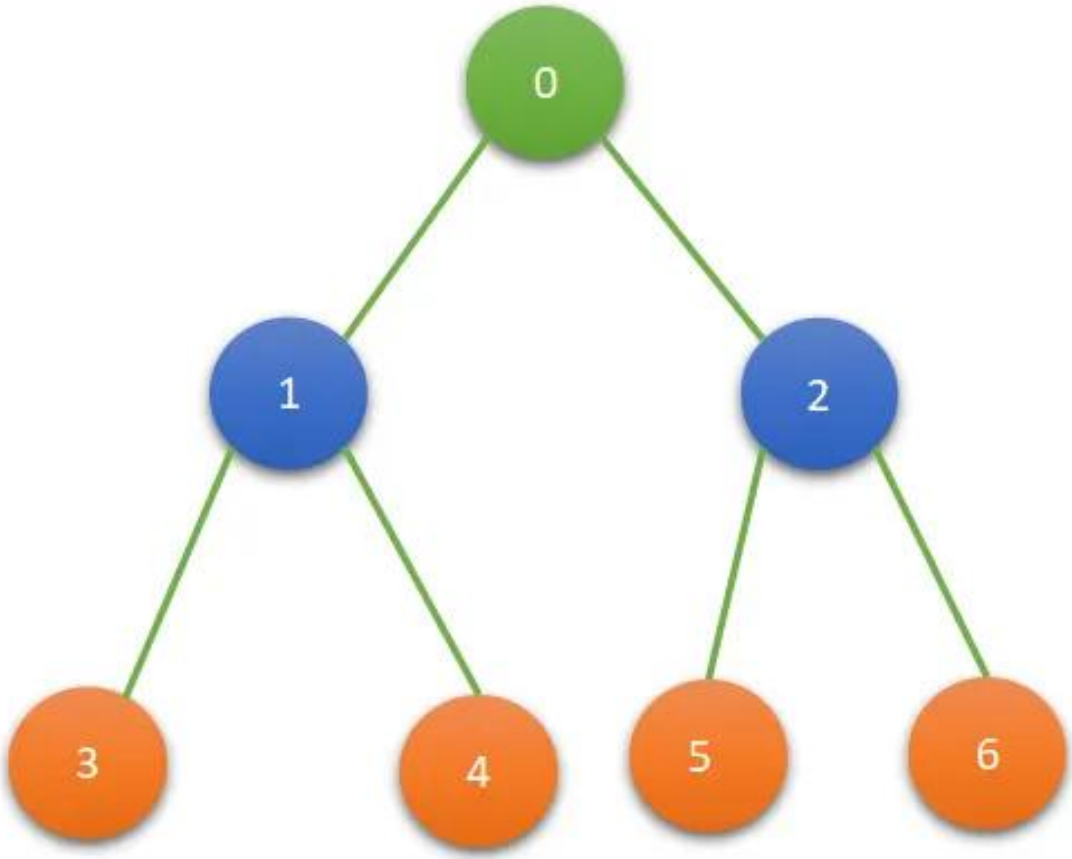


Figure: Perfect Binary Tree

2. Cây nhị phân (Binary tree)

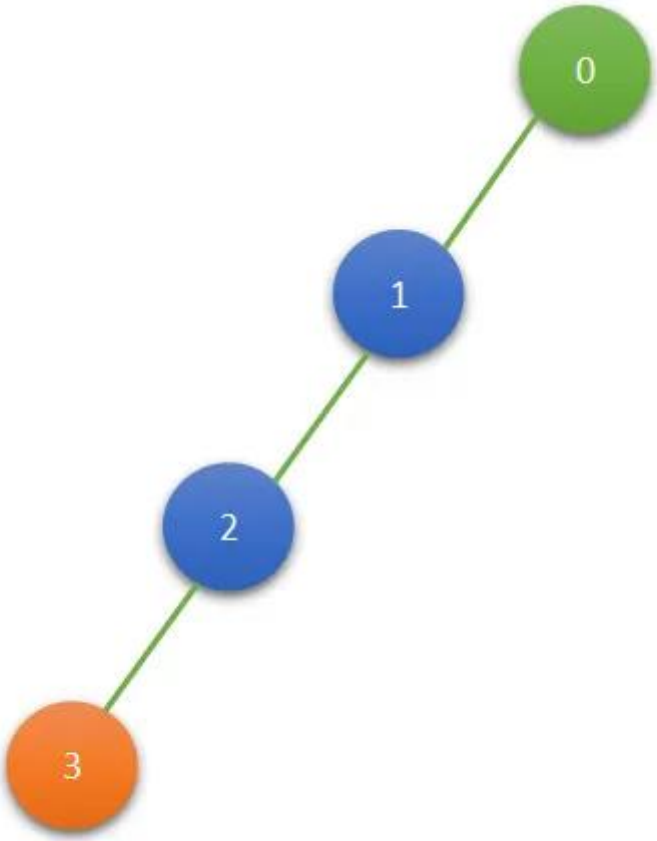


Figure: Pathological Binary Tree

2. Cây nhị phân (Binary tree)

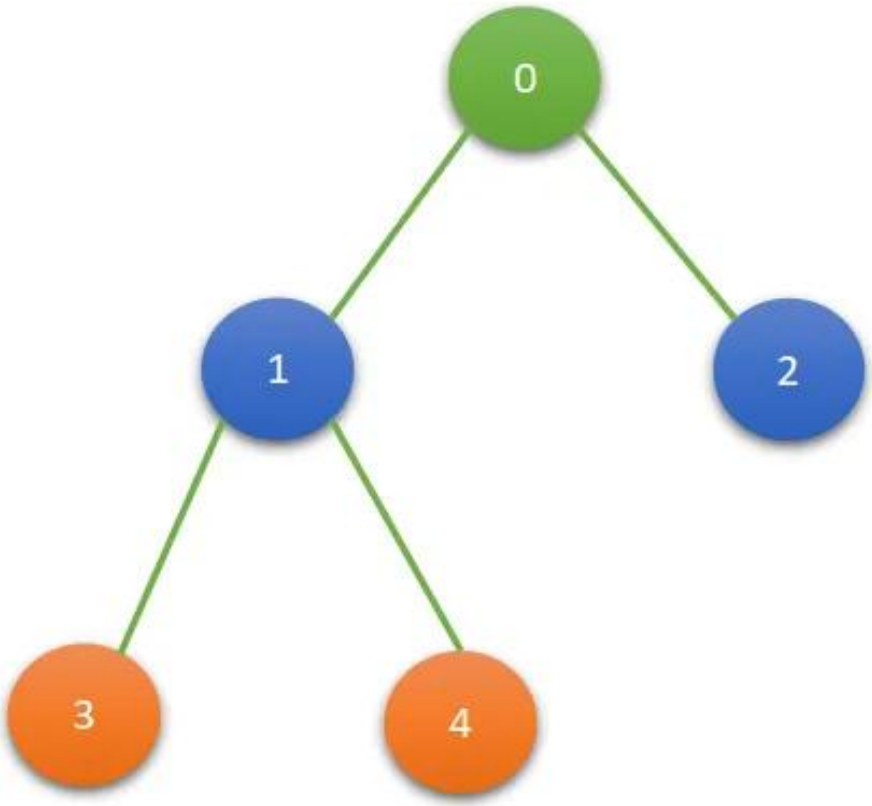


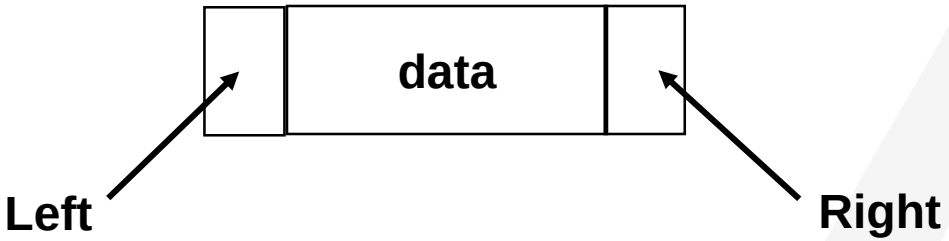
Figure: Balanced Binary Tree(h1=2, h2=1)

Tính chất của cây nhị phân

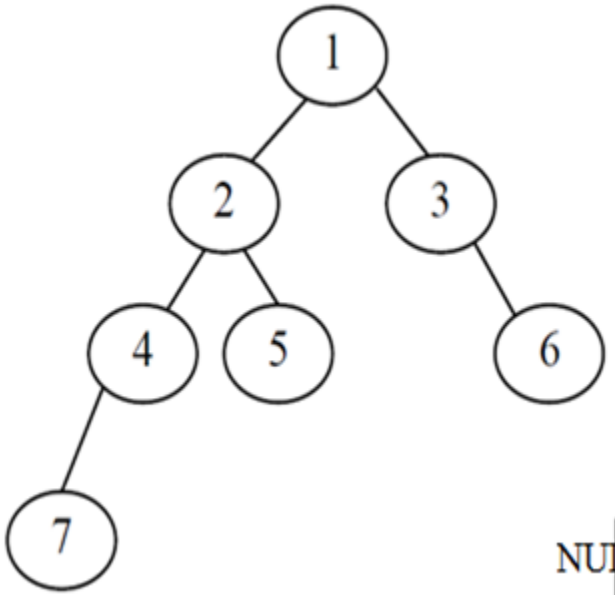
- Số lượng tối đa các nút ở mức i của cây nhị phân là 2^i, tối thiểu là 1 ($i \geq 1$).
- Số lượng tối đa các nút trên một cây nhị phân có chiều cao h là $2^{h+1} - 1$, tối thiểu là h+1 ($h \geq 1$).
- Cây nhị phân hoàn chỉnh có n nút thì chiều cao của nó là h = $log_2(n+1) - 1$.

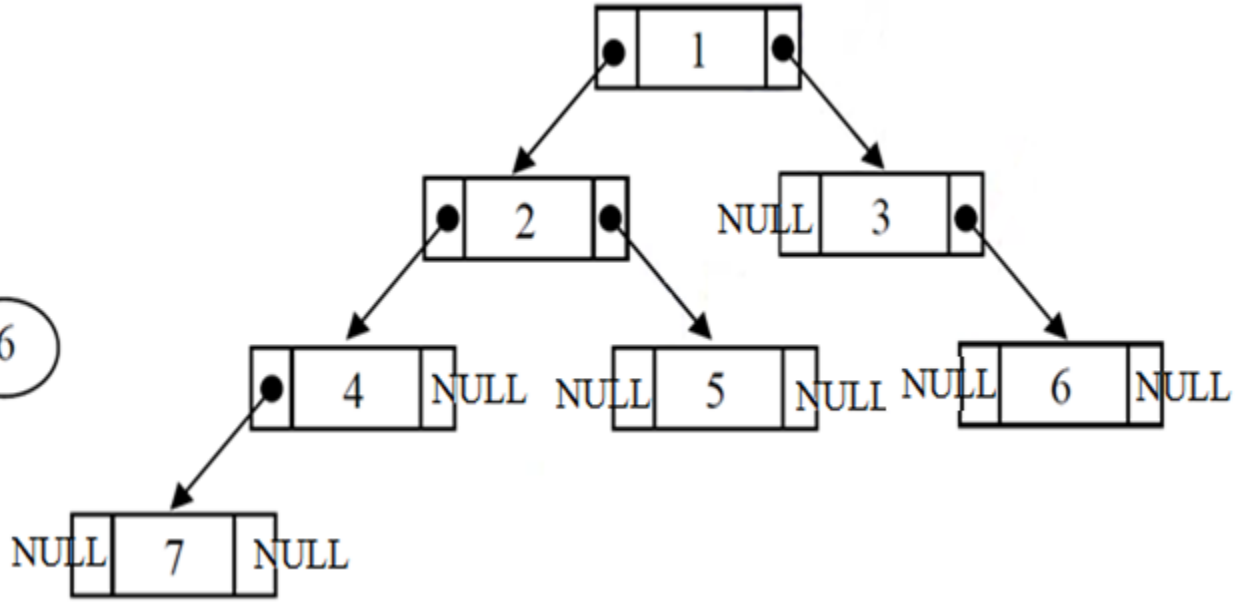
Biểu diễn cây nhị phân

- Mỗi nút của cây nhị phân có tối đa 2 nút con, do vậy sử dụng DSLK để biểu diễn cây nhị phân sẽ hữu hiệu nhất.
- Mỗi nút của cây nhị phân khi đó sẽ có 3 thành phần:
  - ***Thành phần data*** *chứa thông tin về nút.*
  - ***Con trỏ Left*** *trỏ đến nút con bên trái.*
  - ***Con trỏ Right*** *trỏ đến nút con bên phải.*
- Nếu nút có ít hơn 2 nút con thì các thành phần Left hoặc Right sẽ trỏ tới NULL.



Biểu diễn cây nhị phân





Cài đặt cây nhị phân bằng danh sách liên kết.

Biểu diễn cây nhị phân

- **Cấu trúc dữ liệu của nút**

```
struct Node{
    int Data;
    Node *Left, *Right;
};
```

- **Định nghĩa kiểu dữ liệu cây**

```
typedef Node *Tree;
```

- **Khai báo và khởi tạo 1 cây rỗng**

```
Tree T=NULL;
```

Duyệt cây nhị phân

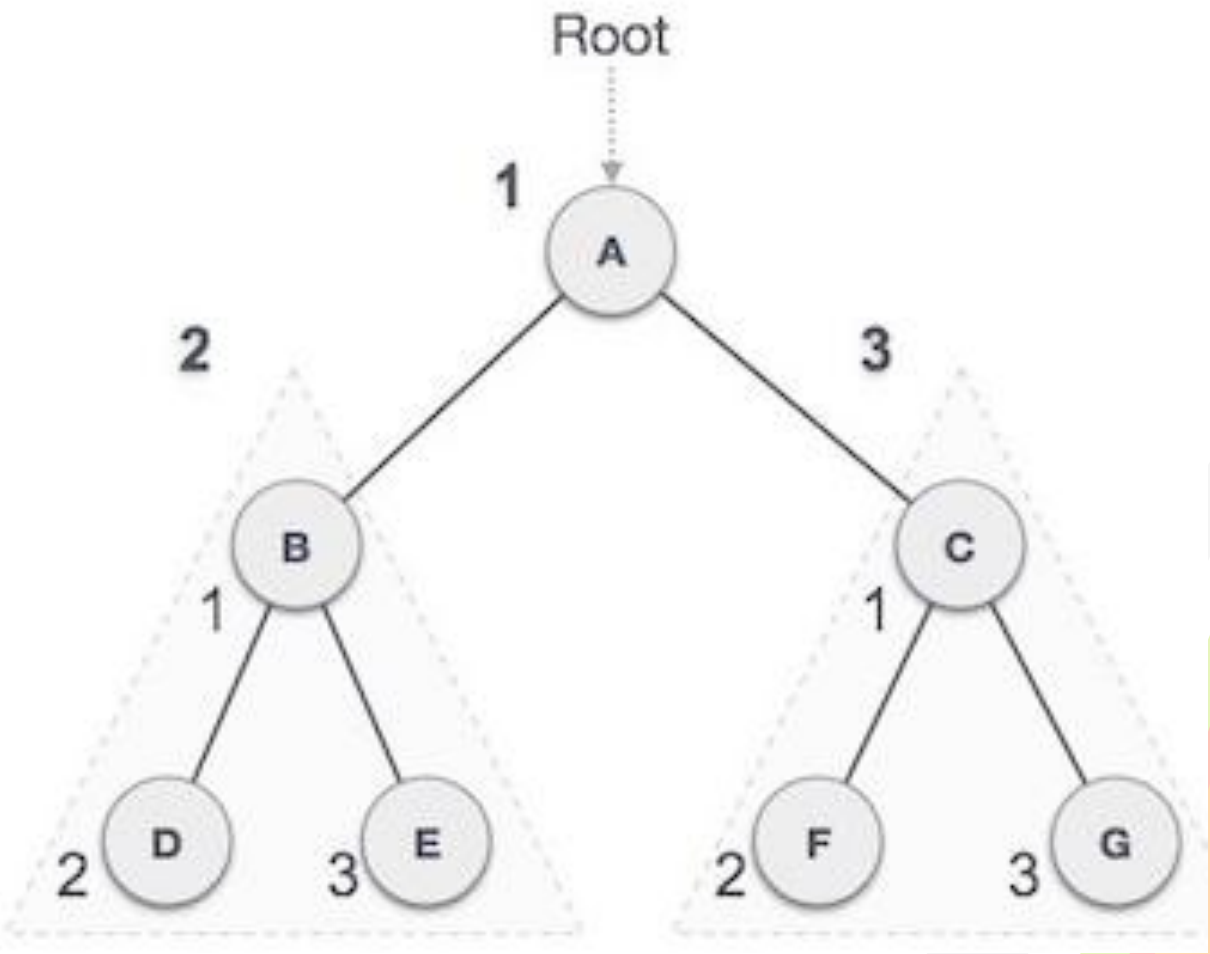
- **Phép duyệt cây nhị phân được chia thành 3 loại:**
  - *Duyệt thứ tự trước (Preorder): NLR hoặc NRL.*
  - *Duyệt thứ tự giữa (Inorder): LNR hoặc RNL.*
  - *Duyệt thứ tự sau (Postorder): LRN hoặc RLN.*

Duyệt cây nhị phân

- **Duyệt trước: NLR**

```
void NLR(Tree  T) {
    if(T != NULL)   {
        cout<<T->Data;
        NLR (T->Left);
        NLR(T->Right);
    }
}
```

Duyệt NLR



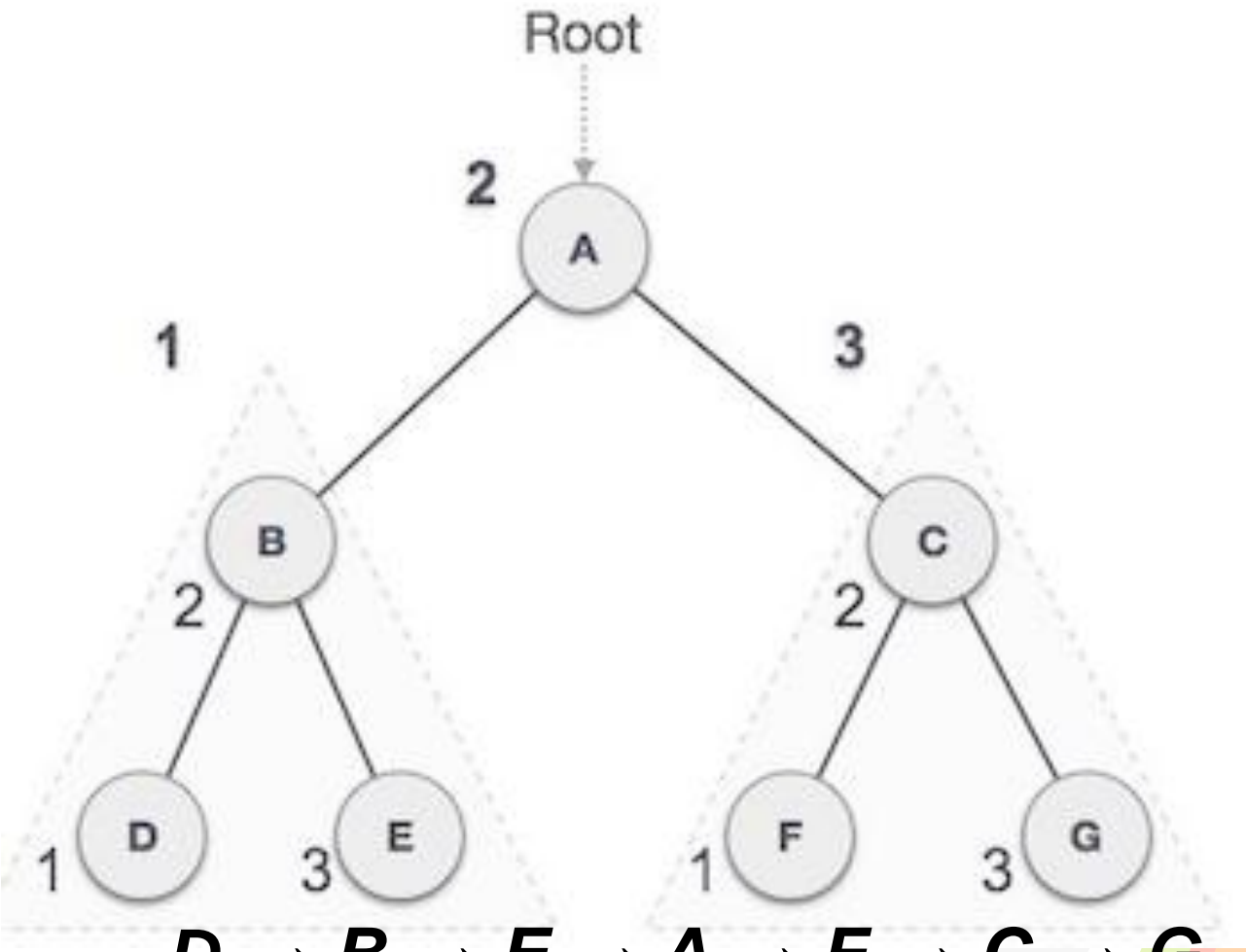
$$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$$

Duyệt cây nhị phân

- **Duyệt giữa: LNR**

```
void LNR(Tree  T) {
    if(T != NULL) {
        LNR(T->Left);
        cout<<T->Data;
        LNR(T->Right);
    }
}
```

Duyệt LNR



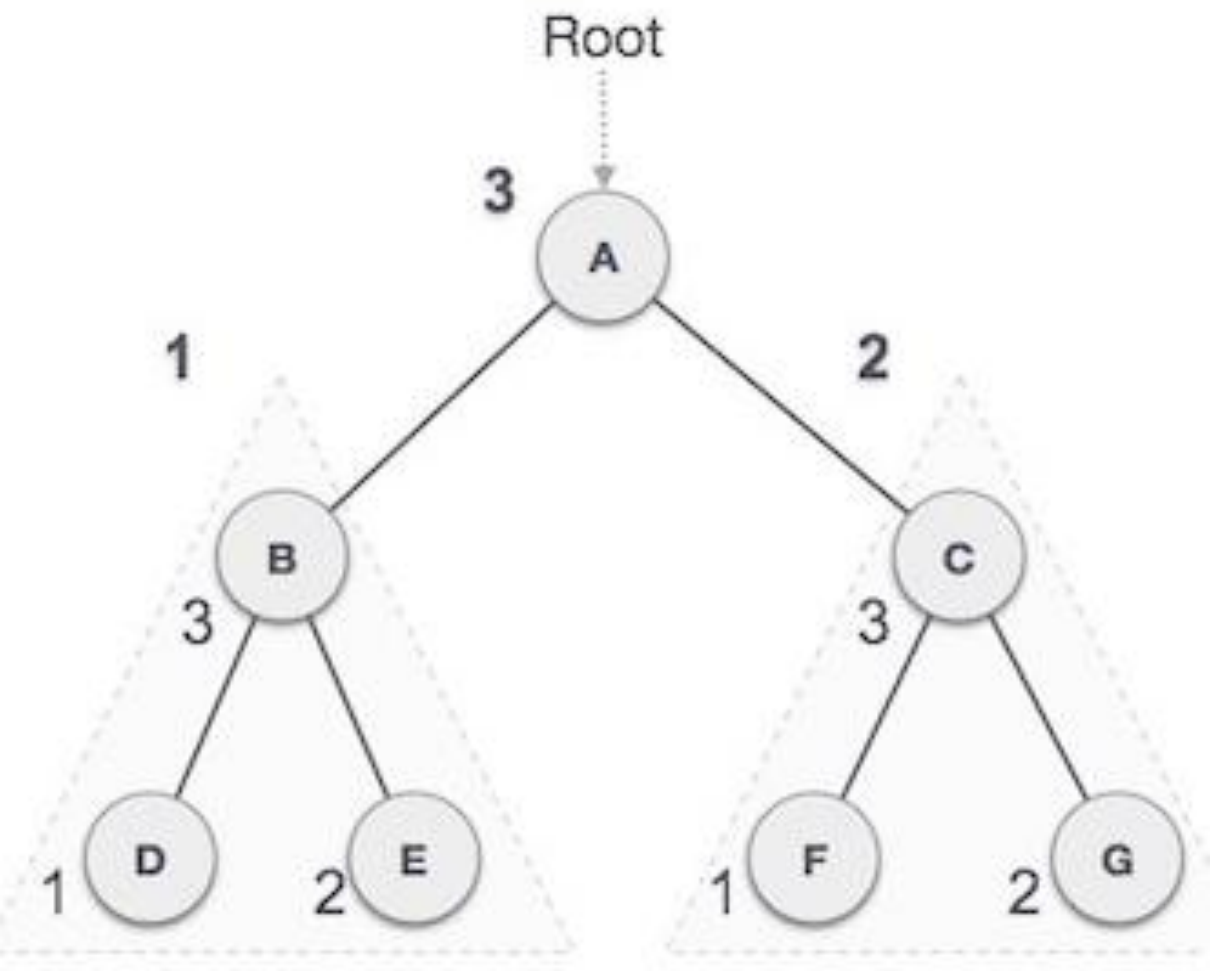
D → B → E → A → F → C → G

Duyệt cây nhị phân

- **Duyệt sau: LRN**

```
void LRN(Tree T) {
    if(T != NULL) {
        LRN(T->Left);
        LRN(T->Right);
        cout<<T->Data;
    }
}
```

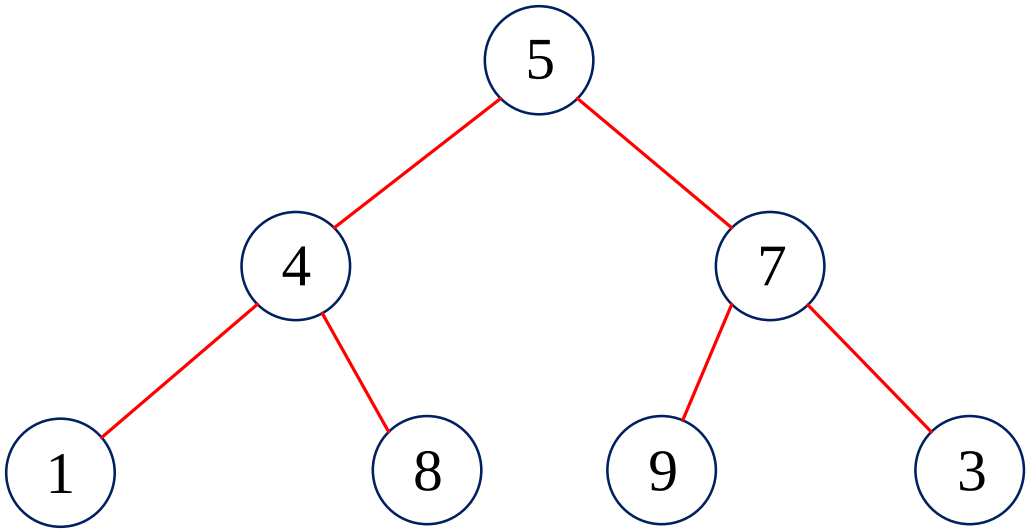
Duyệt LRN



$$D \rightarrow E \rightarrow B \rightarrow F \rightarrow G \rightarrow C \rightarrow A$$

Một số cách biểu diễn cây nhị phân khác

Biểu diễn cây nhị phân bằng mảng:

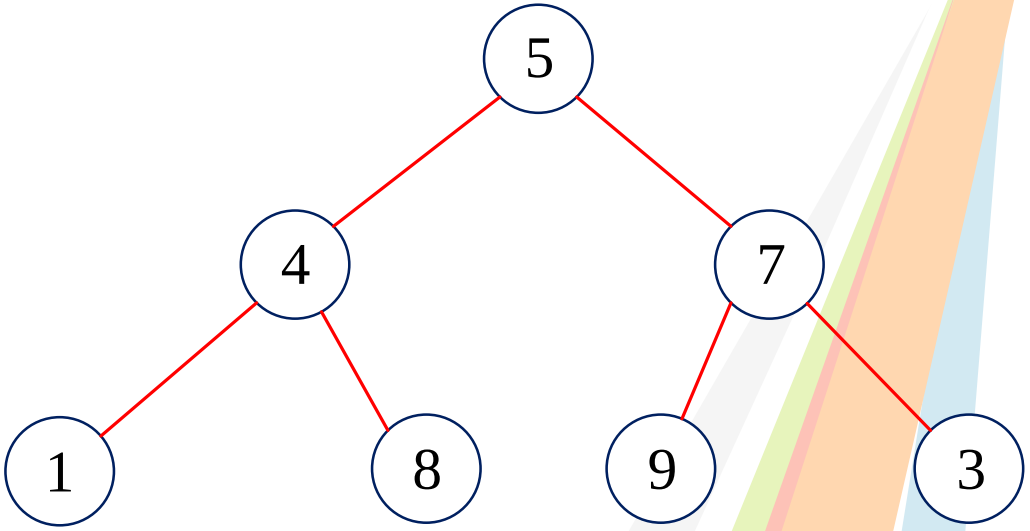


A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]
5	4	7	1	8	9	3

Một số cách biểu diễn cây nhị phân khác

Thêm con trỏ trỏ về nút cha:

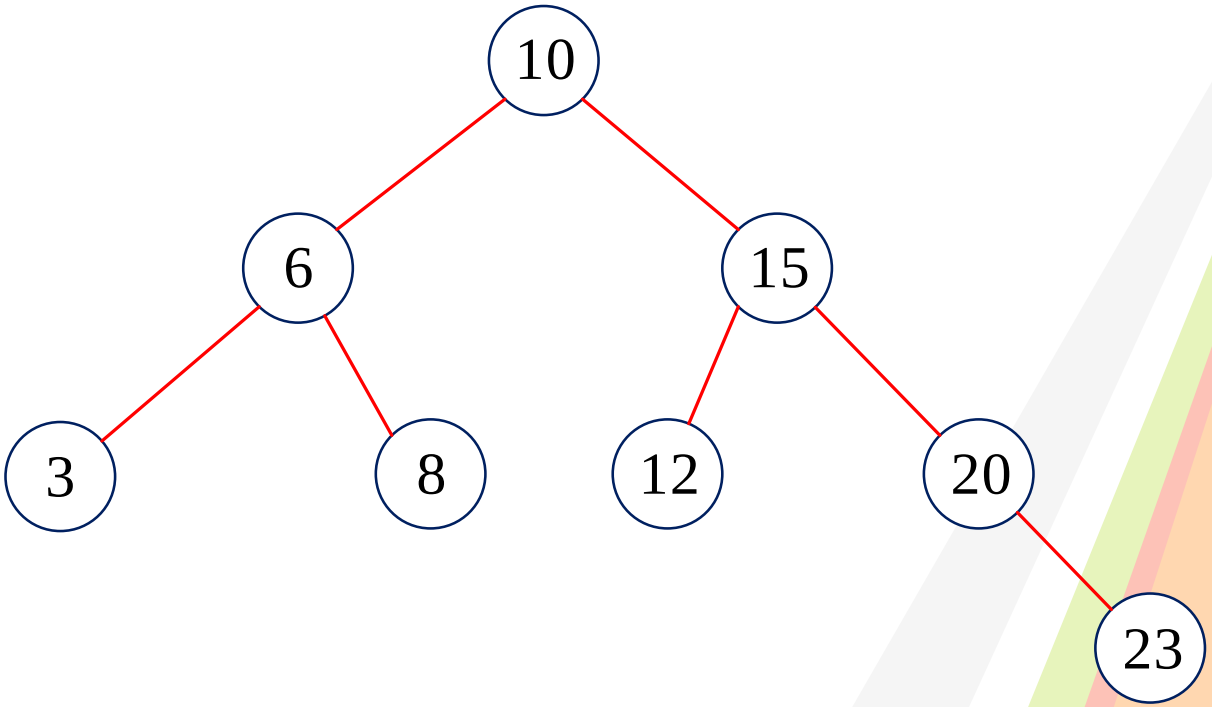
```
struct Node{
   int Data;
   Node *Left;
   Node *Right;
   Node *Parent;
};
```



A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]
5	4	7	1	8	9	3
0	5	5	4	4	7	7

3. Cây nhị phân tìm kiếm (BST)

- **Định nghĩa**: (BST - Binary Search Tree) là cây nhị phân mà khoá tại mỗi nút cây lớn hơn khoá của tất cả các nút thuộc cây con bên trái và nhỏ hơn khoá của tất cả các nút thuộc cây con bên phải.



Khai báo và khởi tạo

- *Cấu trúc của nút*

```
struct Node {
    int Key;//Trường dữ liệu là số nguyên
    Node *Left, *Right;
};
```

- *Định nghĩa kiểu dữ liệu cây*

```
typedef  Node *Tree;
```

- *Khởi tạo cây rỗng*

```
void Create(Tree &T) {
        T=NULL;
}
```

Các thao tác cơ bản trên cây NPTK

- *Tạo 1 nút có trường Key bằng x.*
- *Thêm 1 nút vào cây nhị phân tìm kiếm.*
- *Xoá 1 nút có Key bằng x trên cây.*
- *Tìm 1 nút có khoá bằng x trên cây.*

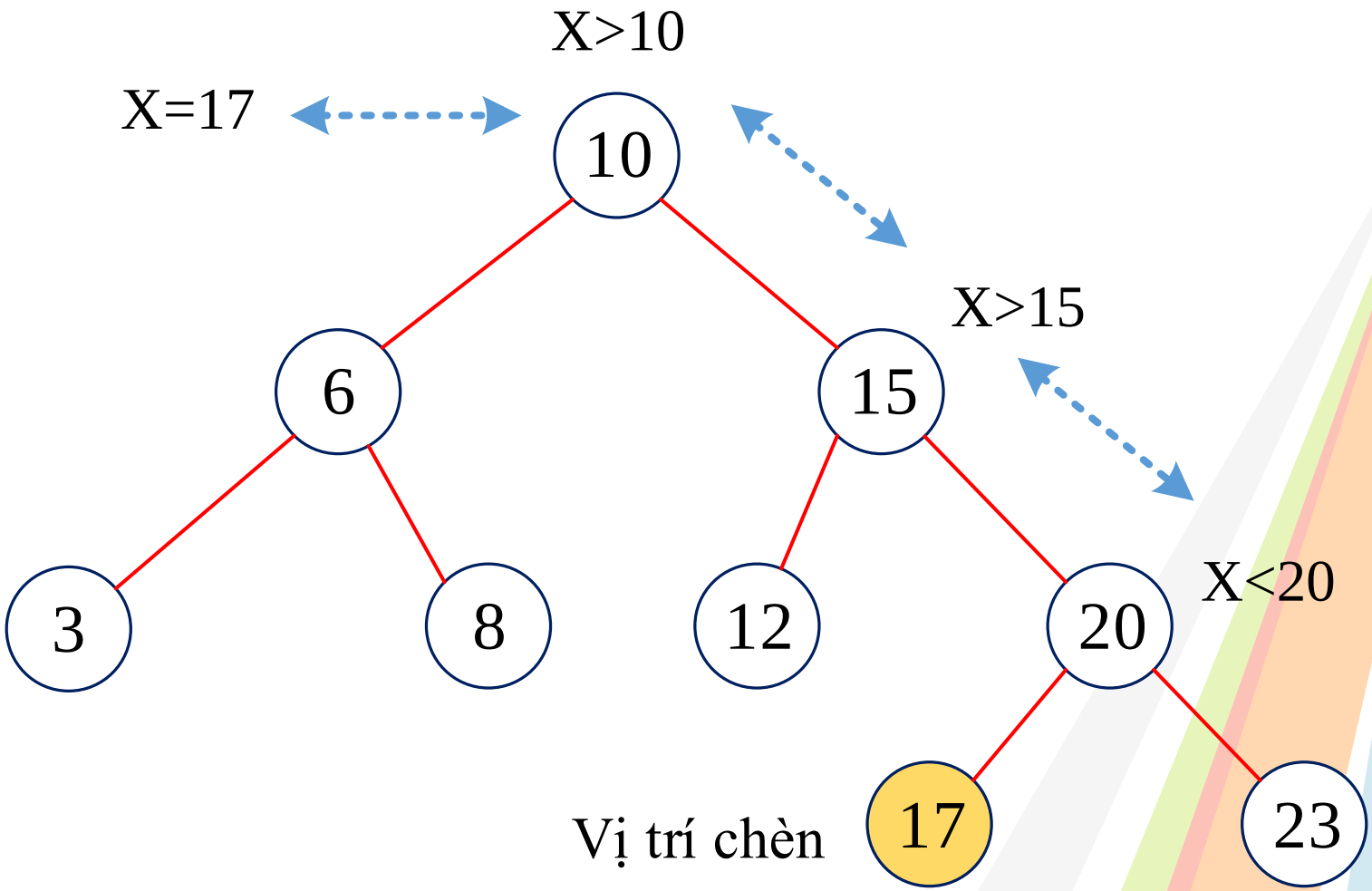
Tạo một nút có khóa bằng x

```
Node* CreateNode(int x) {
      Node *p=new Node;
      p->Key = x;
      p->Left = p->Right =NULL;
      return p;
}
```

Thêm một nút có khóa x vào cây

```
int InsNode(Tree &T, int x) {
    if(T!=NULL) {
        if(T->Key == x)
            return 0;//x trùng
        if(T->Key > x)
            return InsNode(T->Left, x);
        else
            return InsNode(T->Right, x);
    }
    T = CreateNode(x);
    return 1;
}
```

Minh họa thêm phần tử vào cây



Tìm nút có khóa x, không dùng đệ quy

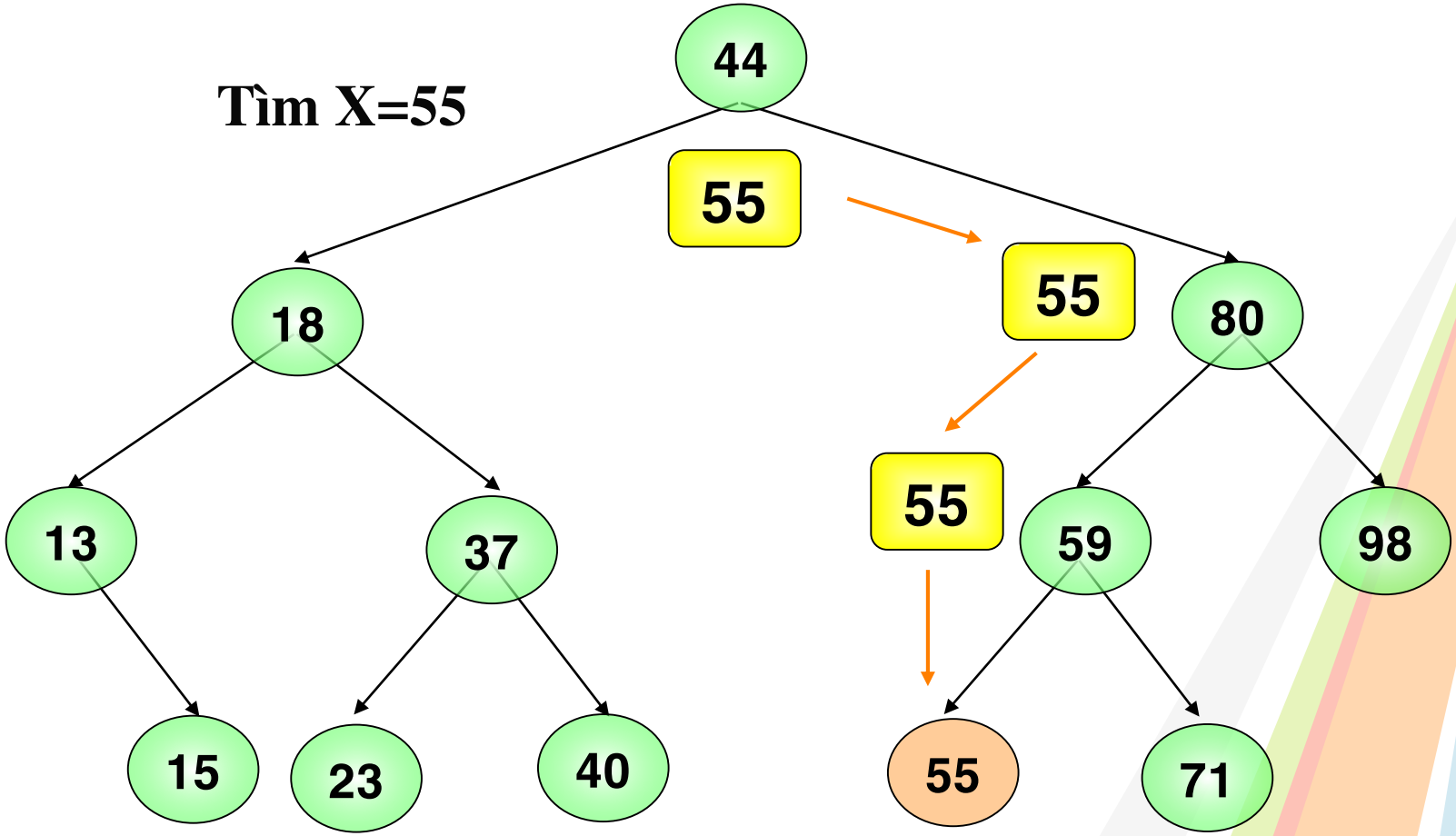
```
Node* Search(Tree T, int x) {
    Node *p = T;
    while(p!=NULL) {
        if(x==p->Key)
            return p; //Tim dc nut p co khoa x
        if(x<p->Key)
            p=p->Left; //Tim tiep o ben trai
        if(x>p->Key)
            p=p->Right; //Tim tiep o ben phai
    }
    return NULL; //Khong tim dc nut nao co khoa x
}
```

Tìm nút có khóa x, dùng đệ quy

```
Node* Search(Tree T, int x) {
      if(T!=NULL) {
         if(x==T->Key)
            return T;
         if(x<T->Key)
            return Search(T->Left, x);
         if(x>T->Key)
            return Search(T->Right, x);
      }
      return NULL; //Khong tim dc nut nao co khoa x
}
```

Minh họa tìm nút

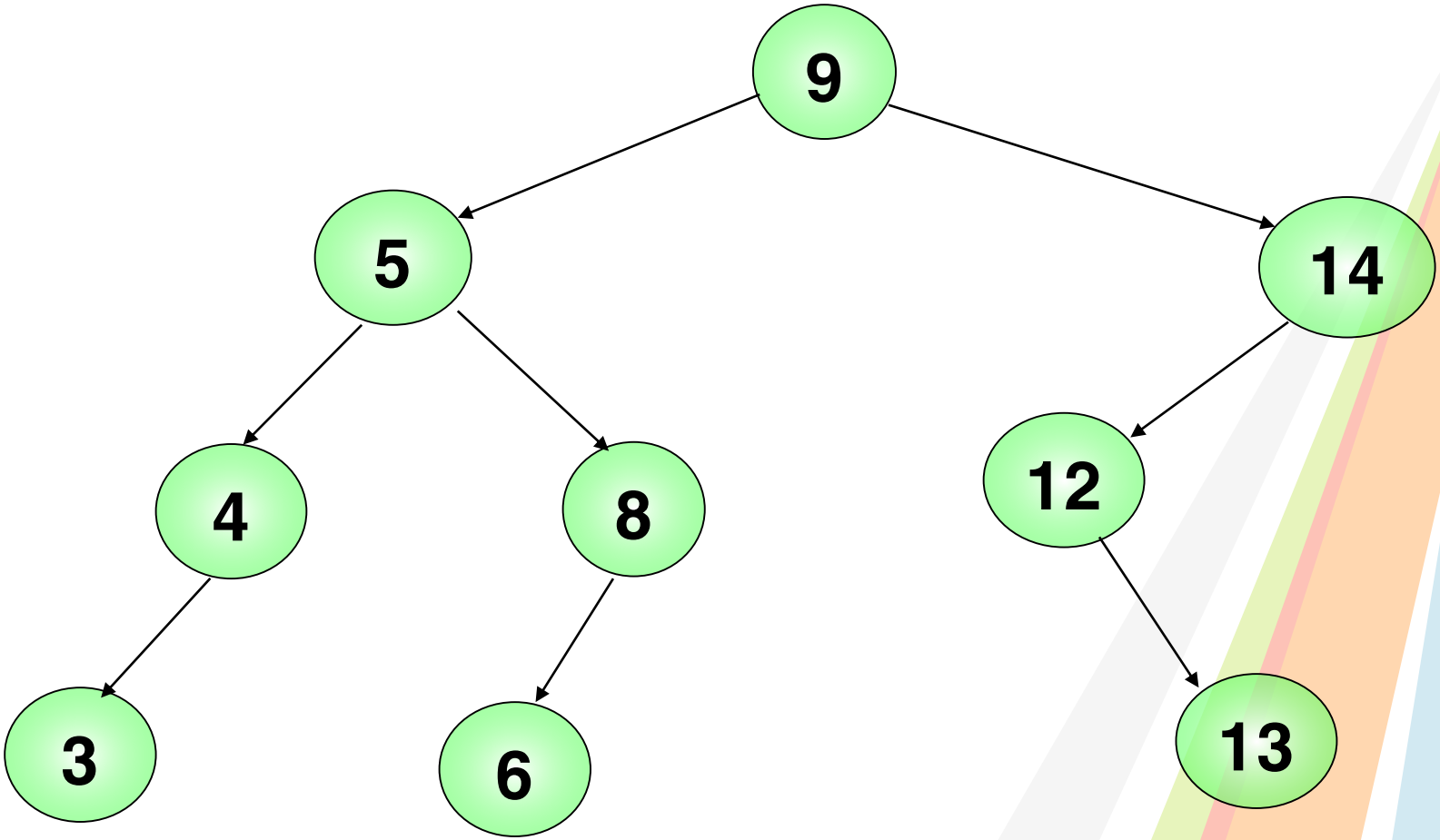
Tìm X=55



Tìm thấy X=55

Minh họa tạo 1 cây NPTK

9, 5, 4, 8, 6, 3, 14,12,13



Hủy nút có khóa x trên cây

Có 3 trường hợp khi hủy 1 nút trên cây:

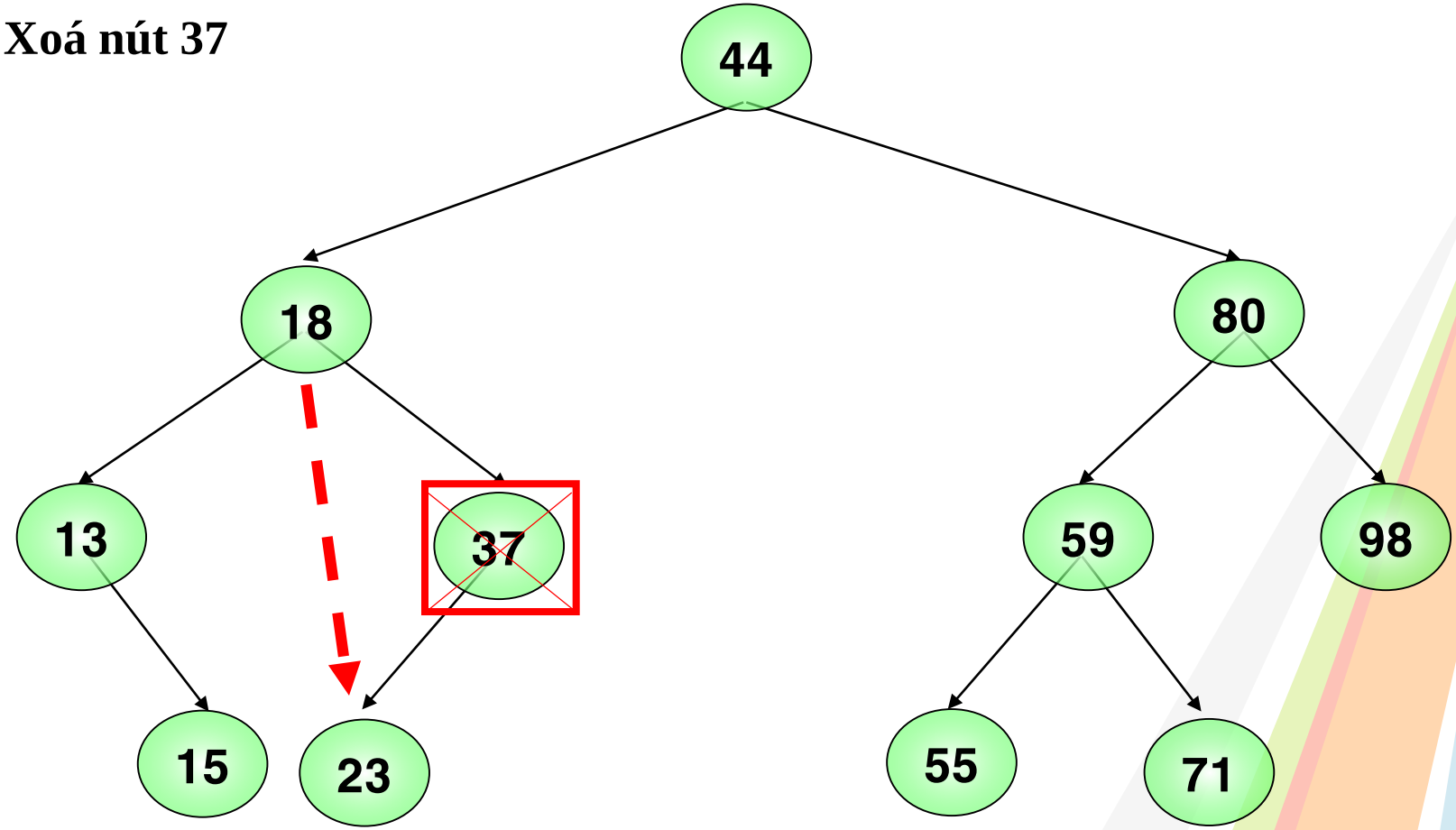
- X là nút lá – *Hủy bình thường mà không ảnh hưởng đến các nút khác trên cây.*
- X chỉ có 1 cây con - *Trước khi xoá x ta móc nối cha của X với con duy nhất của X.*
- X có đầy đủ 2 cây con - *Ta dùng cách xoá gián tiếp.*

Hủy nút có khóa x trên cây

- **Xóa gián tiếp:**
  - Thay vì hủy X ta tìm phần tử thế mạng Y. Nút Y có tối đa 1 cây con.
  - Thông tin lưu tại nút Y sẽ được chuyển lên lưu tại X.
  - Ta tiến hành xoá hủy nút Y (xoá Y giống 2 trường hợp đầu).
  - Cách tìm nút thế mạng Y cho X: Có 2 cách:
    - C1: Nút Y là nút có **khoá nhỏ nhất** (trái nhất) bên **cây con phải** X.
    - C2: Nút Y là nút có **khoá lớn nhất** (phải nhất) bên **cây con trái** của X.

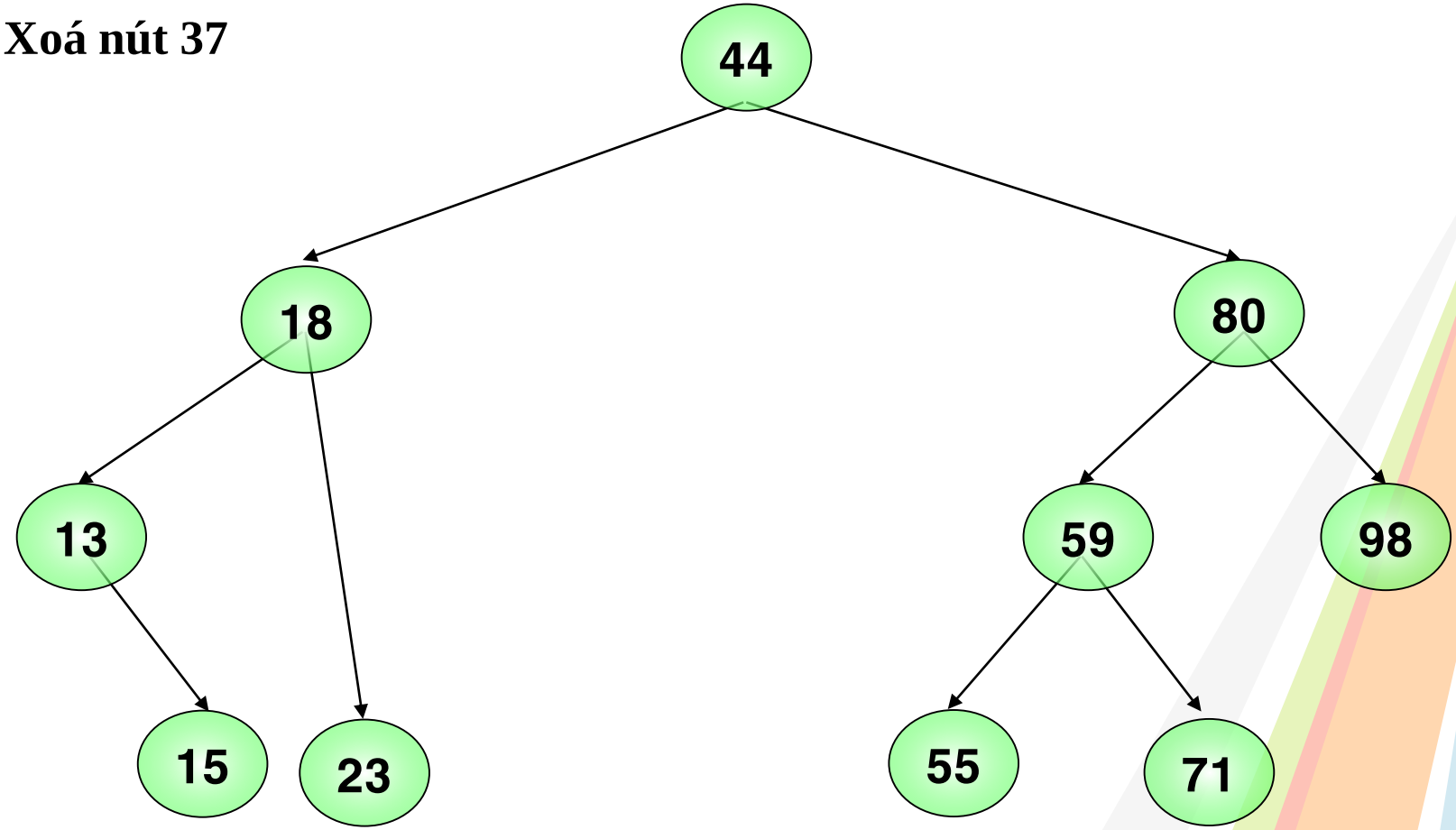
Minh họa hủy nút có 1 cây con

Xoá nút 37



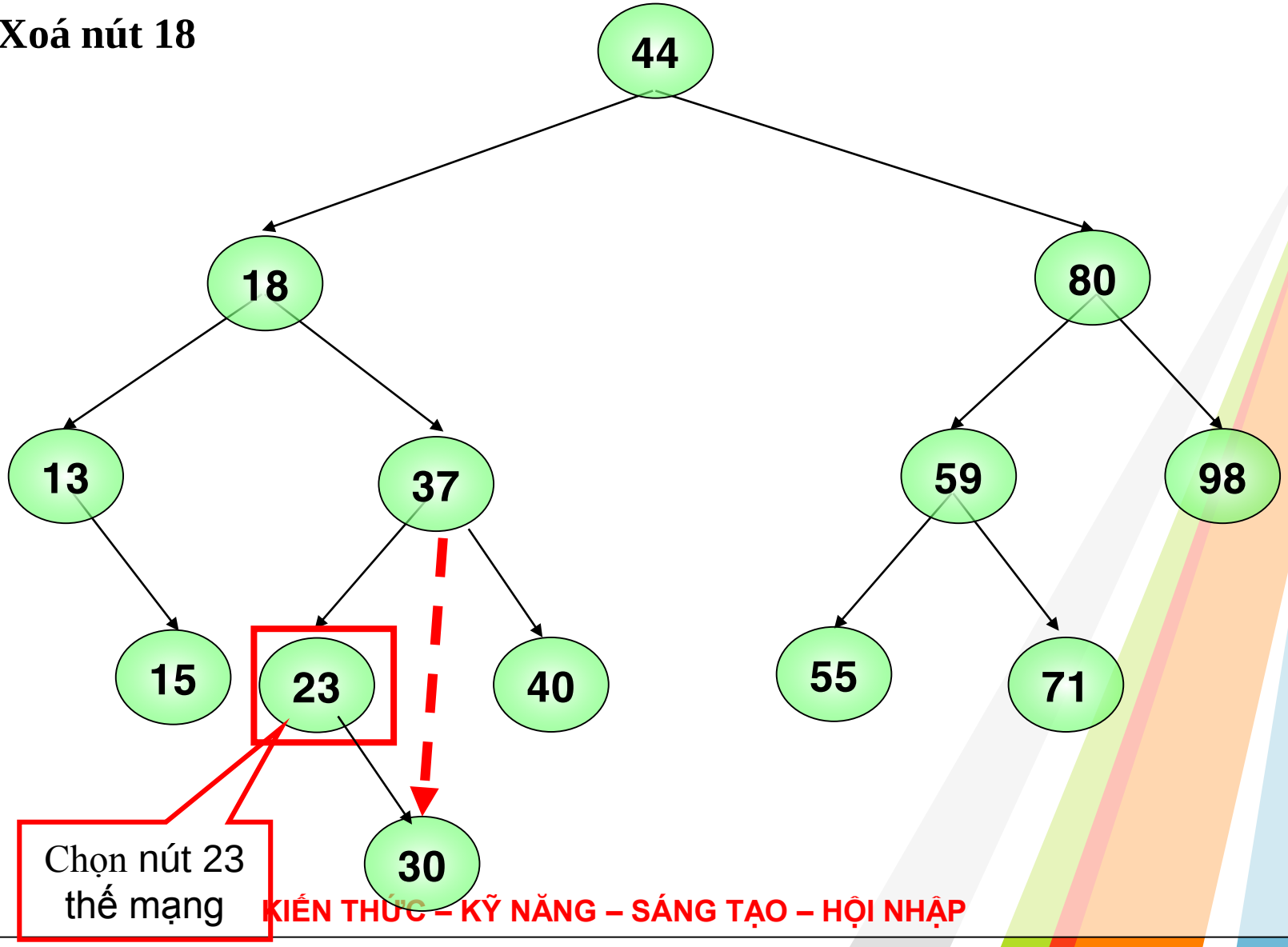
Minh họa hủy nút có 1 cây con

Xoá nút 37



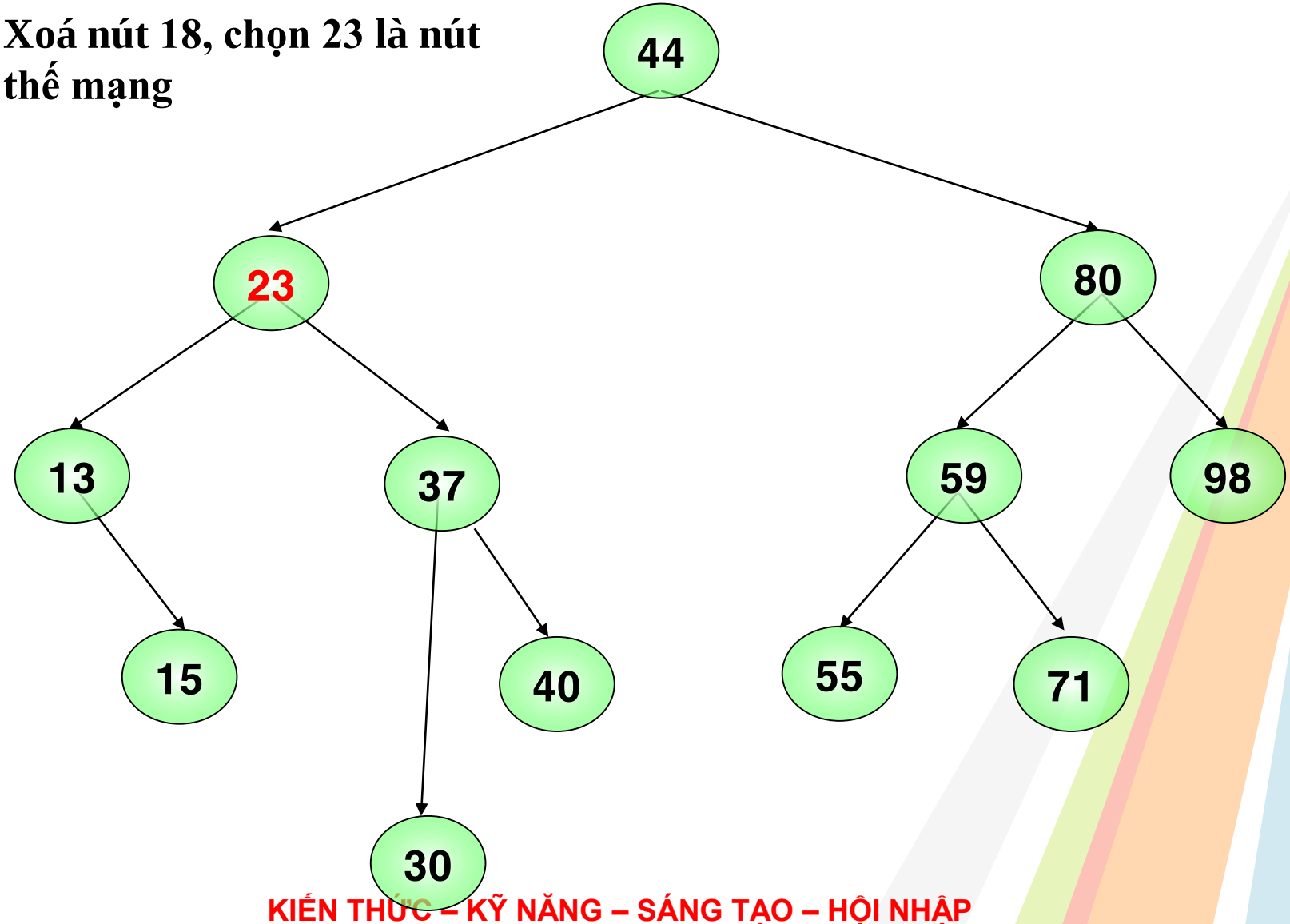
Minh họa hủy nút có 2 cây con

Xoá nút 18



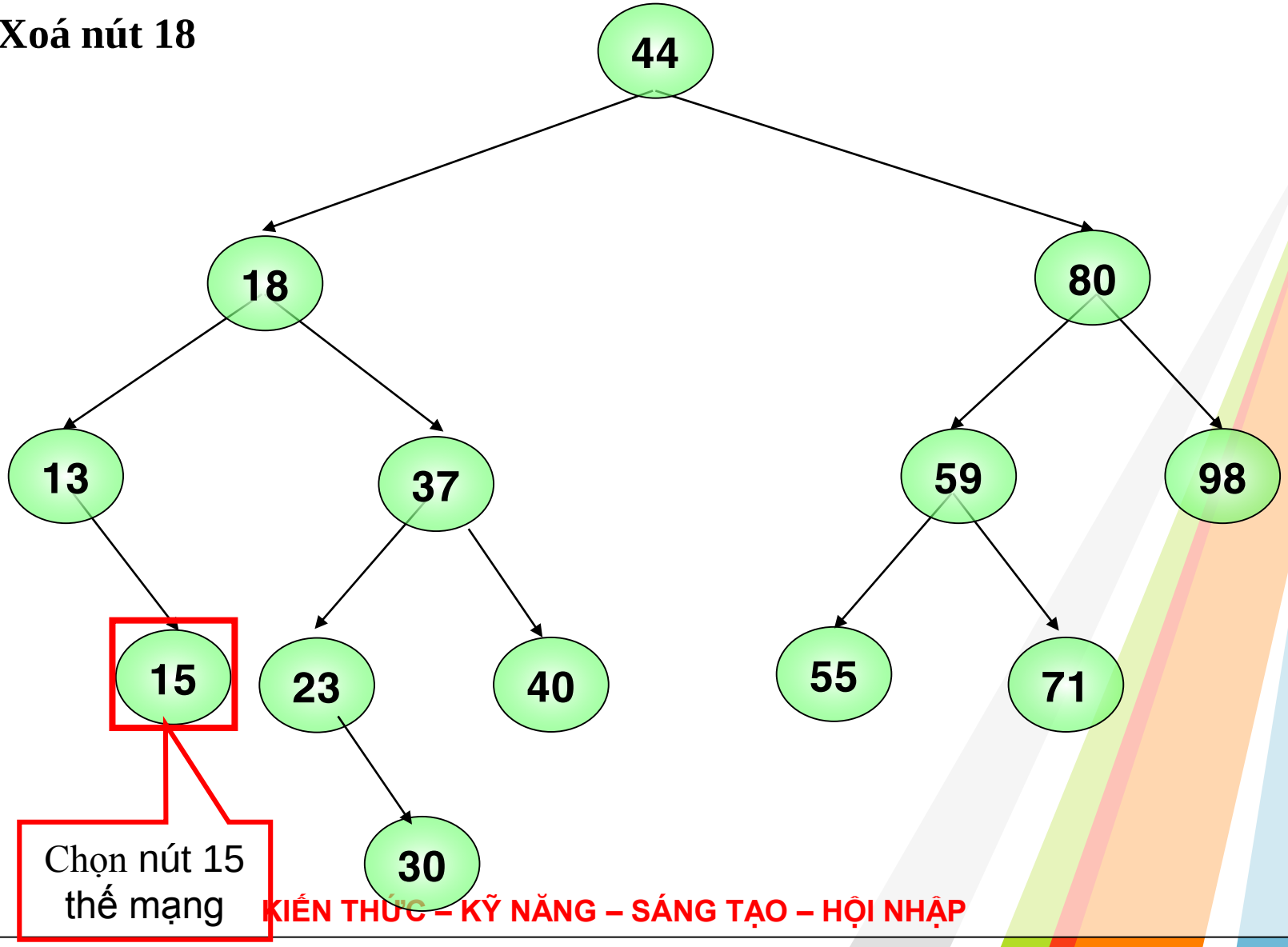
Minh họa hủy nút có 1 cây con

Xoá nút 18, chọn 23 là nút thế mạng



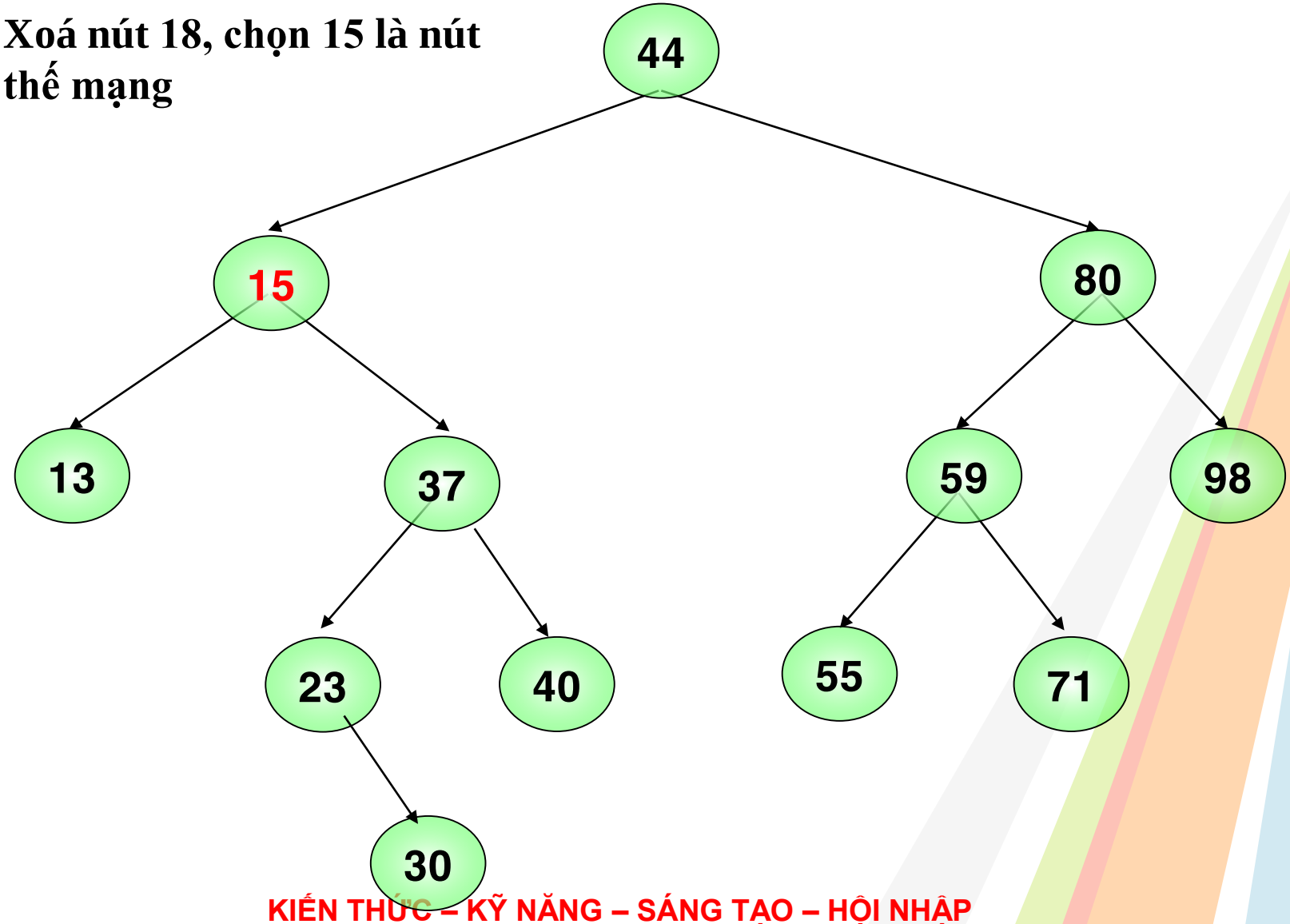
Minh họa hủy nút có 1 cây con

Xoá nút 18



Minh họa hủy nút có 1 cây con

Xoá nút 18, chọn 15 là nút thế mạng



Cài đặt thao tác xóa nút có Key x

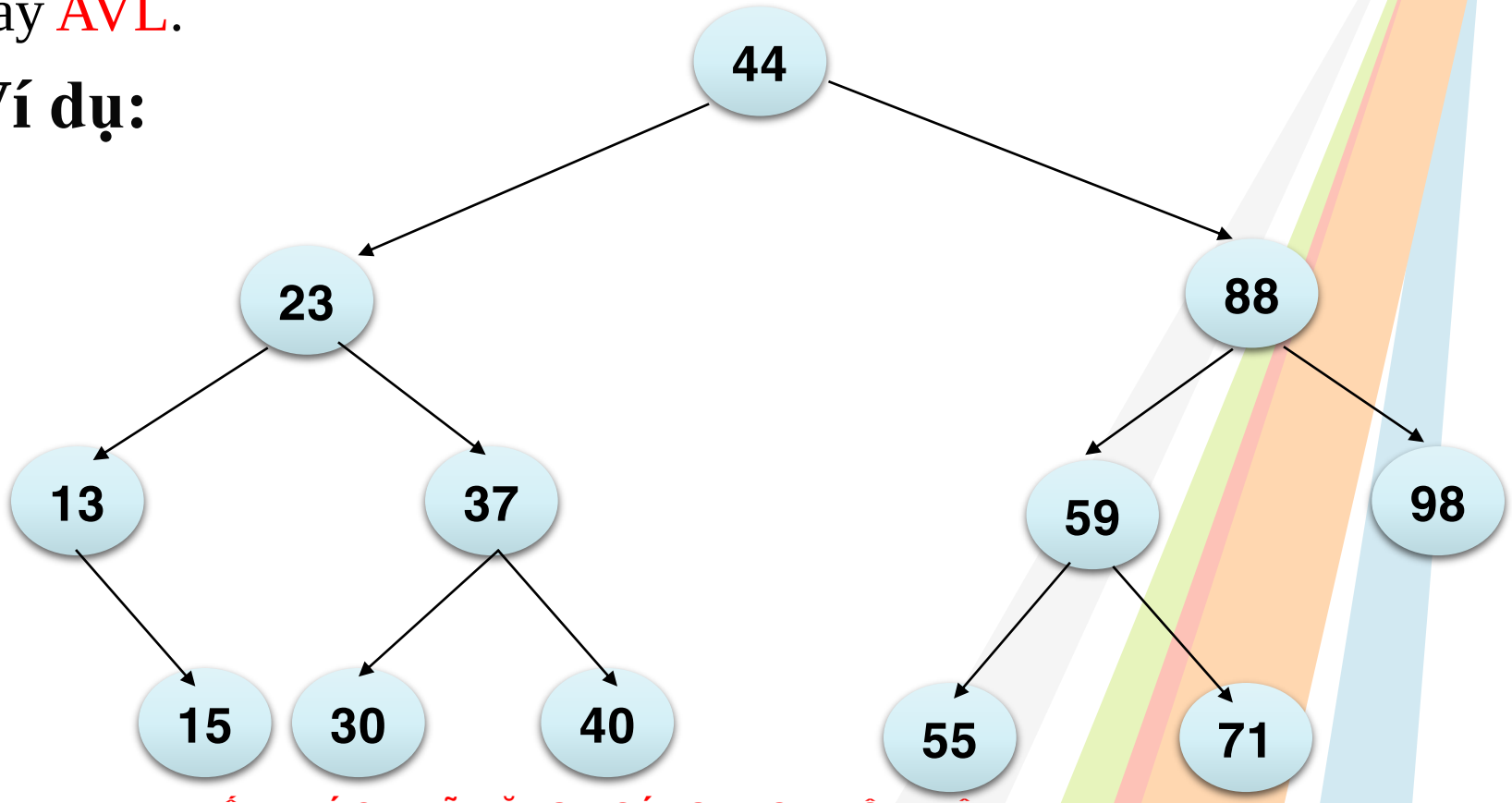
```
int DelNode(Tree &T, int x) {
    if (T==NULL) return 0;
    else if (T->Key>x) return DelNode(T->Left, x);
    else if (T->Key<x) return DelNode(T->Right, x);
    else {//T->Key == x
        Node *P=T;
        if (T->Left==NULL)
            T=T->Right;//Node chi co cay con phai
        else if (T->Right==NULL)
                T=T->Left; // Node chi co cay con trai
            else { // Node co ca 2 con
                Node *S = T,
                *Q=S->Left;
                 //S la cha cua Q, Q la Node phai nhat cua
cay con trai cua P
```

Cài đặt thao tác xóa nút có Key x

```
            while (Q->Right!=NULL) {
                S=Q;
               Q=Q->Right;
            }
            P->Key=Q->Key;
            S->Right=Q->Left;
            delete Q;
        }
    }
    return 1;
}
```

4. Cây NPTK cân bằng (AVL)

- **Định nghĩa:** là cây mà tại mỗi nút của nó độ cao của cây con trái và của cây con phải chênh lệch không quá một.Do G.M. **A**delsen **V**elskii và E.M. **L**endis đưa ra năm 1962, đặt tên là cây AVL.
- **Ví dụ:**



CTDL của cây NPTK cân bằng

- Chỉ số cân bằng = độ lệch giữa cây trái và cây phải của một nút.
- Các giá trị hợp lệ của CSCB:
  - CSCB(p) = 0 ⇔ Độ cao cây trái (p) = Độ cao cây phải (p).
  - CSCB(p) = 1 ⇔ Độ cao cây trái (p) < Độ cao cây phải (p).
  - CSCB(p) = -1⇔ Độ cao cây trái (p) > Độ cao cây phải (p).

CTDL của cây NPTK cân bằng

Khai báo một nút

```
struct NODE
{
        int key;
        int height;
        NODE* pLeft;
        NODE* pRight;
};
typedef NODE* TREE;
```

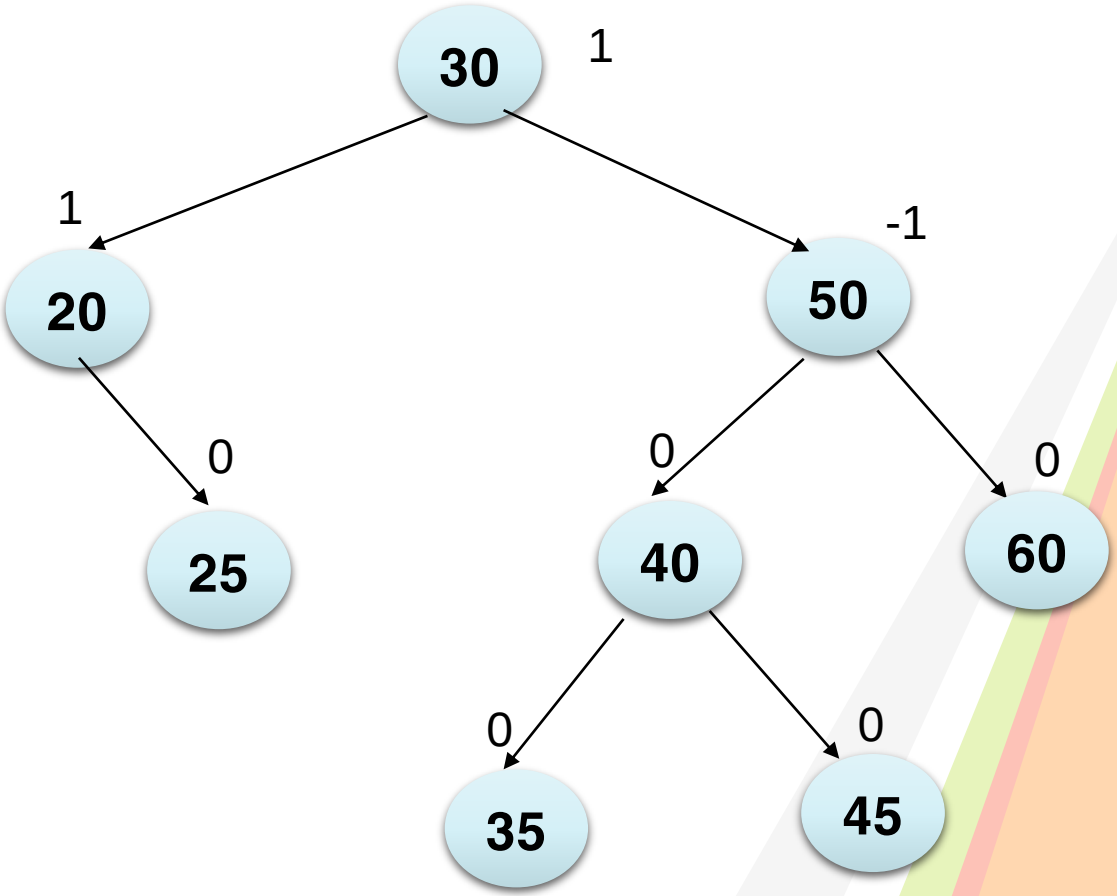
CTDL của cây NPTK cân bằng

Tạo một nút

```
NODE* create_node(int x)
{
        NODE *p = new NODE;
        p->key = x;
        p->height = 1;
        p->pLeft = NULL;
        p->pRight = NULL;
        return p;
}
```

CTDL của cây NPTK cân bằng

Hệ số cân bằng các nút trong cây AVL



Quay trái – quay phải tại 1 node

1) Left Rotation

2) Right Rotation

```
T1, T2 and T3 are subtrees of the tree
rooted with y (on the left side) or x (on
the right side)
     y                               x
    / \     Right Rotation          /  \
   x   T3   - - - - - - - >        T1   y
  / \       < - - - - - - -            / \
 T1  T2     Left Rotation            T2  T3
Keys in both of the above trees follow the
following order
 keys(T1) < key(x) < keys(T2) < key(y) < keys(T3)
So BST property is not violated anywhere.
```

Hàm quay phải tại 1 node

```
NODE* rotation_right(TREE &y)
{
        NODE* x = y->pLeft;
        NODE* T2 = x->pRight;
        //Perform rotation
        x->pRight = y;
        y->pLeft = T2;
        // Update heights
        y->height = max(height_tree(y->pLeft), height_tree(y->pRight)) + 1;
        x->height = max(height_tree(x->pLeft), height_tree(x->pRight)) + 1;
        // Return new root
        return x;
}
```

Hàm quay trái tại 1 node

```
NODE* rotation_left(TREE &x)
{
        NODE* y = x->pRight;
        NODE* T2 = y->pLeft;
        //Perform rotation
        y->pLeft = x;
        x->pRight = T2;
        // Update heights
        x->height = max(height_tree(x->pLeft), height_tree(x->pRight)) + 1;
        y->height = max(height_tree(y->pLeft), height_tree(y->pRight)) + 1;
        // Return new root
        return y;
}
```

Trường hợp mất cân bằng do lệch trái trái

a) Left Left Case

```
T1, T2, T3 and T4 are subtrees.
         z                                      y
        / \                                   /   \
       y   T4      Right Rotate (z)          x      z
      / \          - - - - - - - - ->      /  \    /  \
     x   T3                               T1  T2  T3  T4
    / \
  T1   T2
```

Trường hợp mất cân bằng do lệch trái phải

b) Left Right Case

```
     z                           z                           x
    / \                         /   \                       /  \
   y   T4  Left Rotate (y)      x    T4  Right Rotate(z)    y      z
  / \      - - - - - - - - ->  / \      - - - - - - - - -> / \    / \
T1   x                        y    T3                    T1  T2 T3  T4
    / \                      / \
  T2   T3                   T1   T2
```

Trường hợp mất cân bằng do lệch phải phải

c) Right Right Case

```
   z                                      y
  /  \                                  /   \
 T1   y     Left Rotate(z)            z     x
     /  \   - - - - - - - ->         / \   / \
    T2   x                          T1  T2 T3  T4
        / \
      T3  T4
```

Trường hợp mất cân bằng do lệch phải trái

d) Right Left Case

```
   z                           z                           x
  / \                         / \                         /  \
T1   y   Right Rotate (y)   T1   x      Left Rotate(z)   z      y
    / \  - - - - - - - - ->    / \   - - - - - - - - -> / \    / \
   x   T4                     T2   y                   T1  T2  T3  T4
  / \                            / \
T2   T3                         T3   T4
```

Các thao tác trên cây cân bằng

- Khi thêm hay xoá 1 nút trên cây, có thể làm cho cây mất tính cân bằng, khi ấy ta phải tiến hành cân bằng lại.
- Cây có khả năng mất cân bằng khi thay đổi chiều cao:
  - *Thêm bên trái -> lệch nhánh trái.*
  - *Thêm bên phải -> lệch nhánh phải.*
  - *Hủy bên phải -> lệch nhánh trái.*
  - *Hủy bên trái -> lệch nhánh phải.*
- Cân bằng lại cây : Tìm cách bố trí lại cây sao cho chiều cao 2 cây con cân đối:
  - *Kéo nhánh cao bù cho nhánh thấp.*
  - *Phải bảo đảm cây vẫn là nhị phân tìm kiếm.*

Thêm nút vào cây NPTK CB

- Thêm bình thường như trường hợp cây NPTK.
- Nếu cây tăng trưởng chiều cao.
  - Lần ngược về gốc để phát hiện nút bị mất cân bằng.
  - Tiến hành cân bằng lại nút đó bằng thao tác cân bằng thích hợp.
- Việc cân bằng lại chỉ cần thực hiện 1 lần nơi mất cân bằng.

Thêm nút vào cây NPTK CB

Steps to follow for insertion

Let the newly inserted node be w.

1) Perform standard BST insert for w.

2) Starting from w, travel up and find the first unbalanced node. Let z be the first unbalanced node, y be the child of z that comes on the path from w to z and x be the grandchild of z that comes on the path from w to z.

Thêm nút vào cây NPTK CB

3) Re-balance the tree by performing appropriate rotations on the subtree rooted with z. There can be 4 possible cases that needs to be handled as x, y and z can be arranged in 4 ways. Following are the possible 4 arrangements:

a) y is left child of z and x is left child of y (Left Left Case).

b) y is left child of z and x is right child of y (Left Right Case).

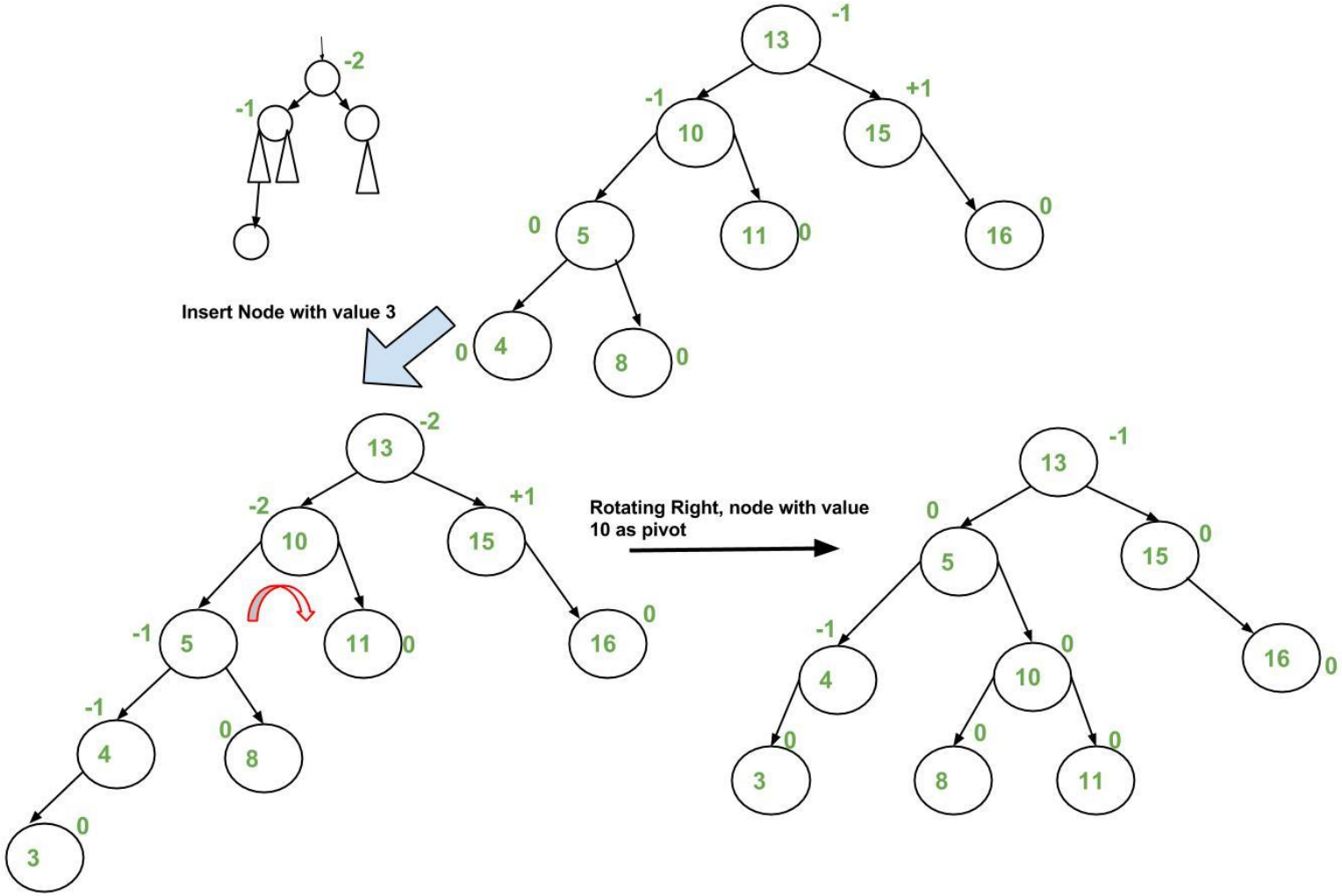
c) y is right child of z and x is right child of y (Right Right Case) .

d) y is right child of z and x is left child of y (Right Left Case).

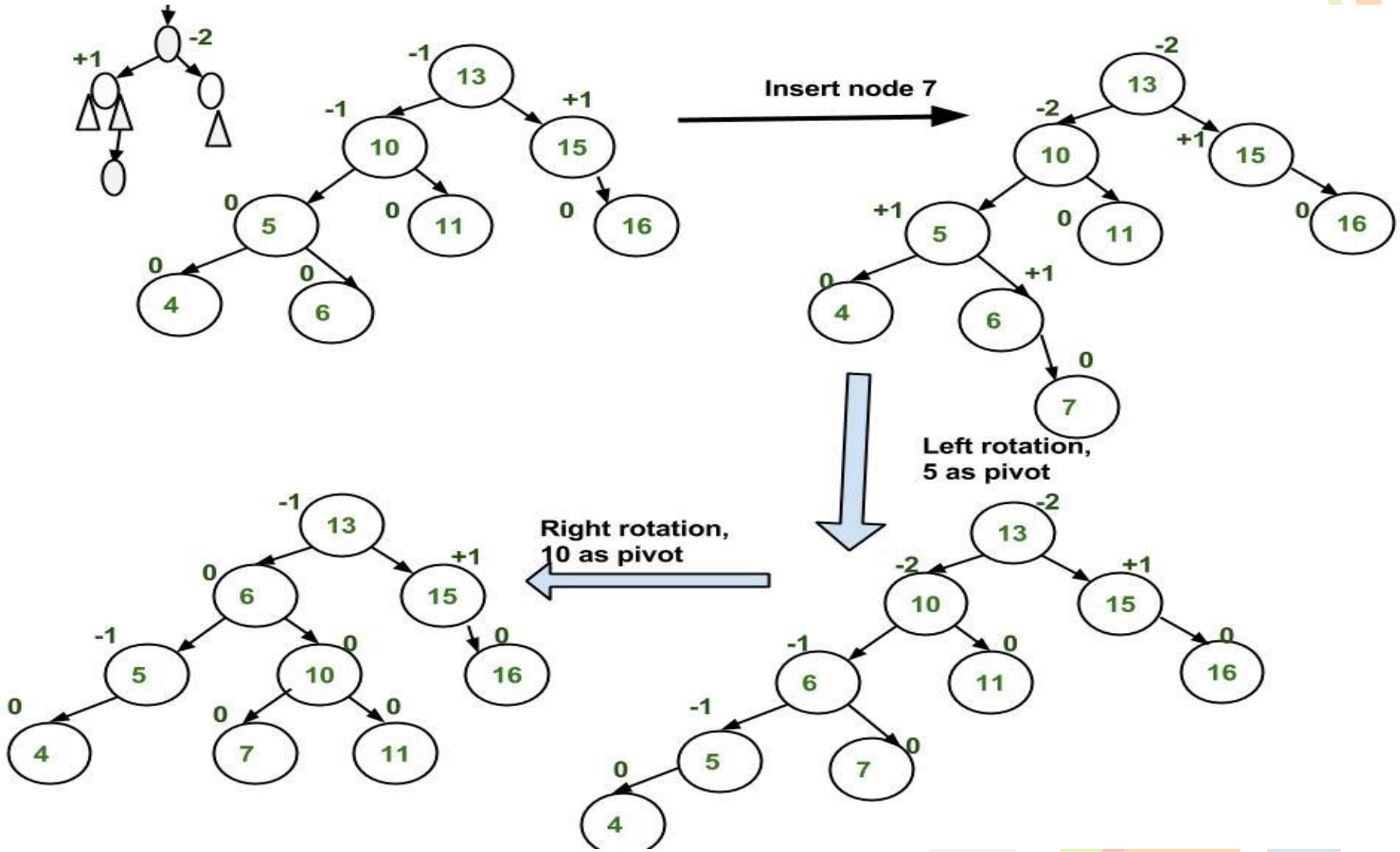
Thêm nút vào cây NPTK CB

Following are the operations to be performed in above mentioned 4 cases. In all of the cases, we only need to re-balance the subtree rooted with z and the complete tree becomes balanced as the height of subtree (After appropriate rotations) rooted with z becomes same as it was before insertion.

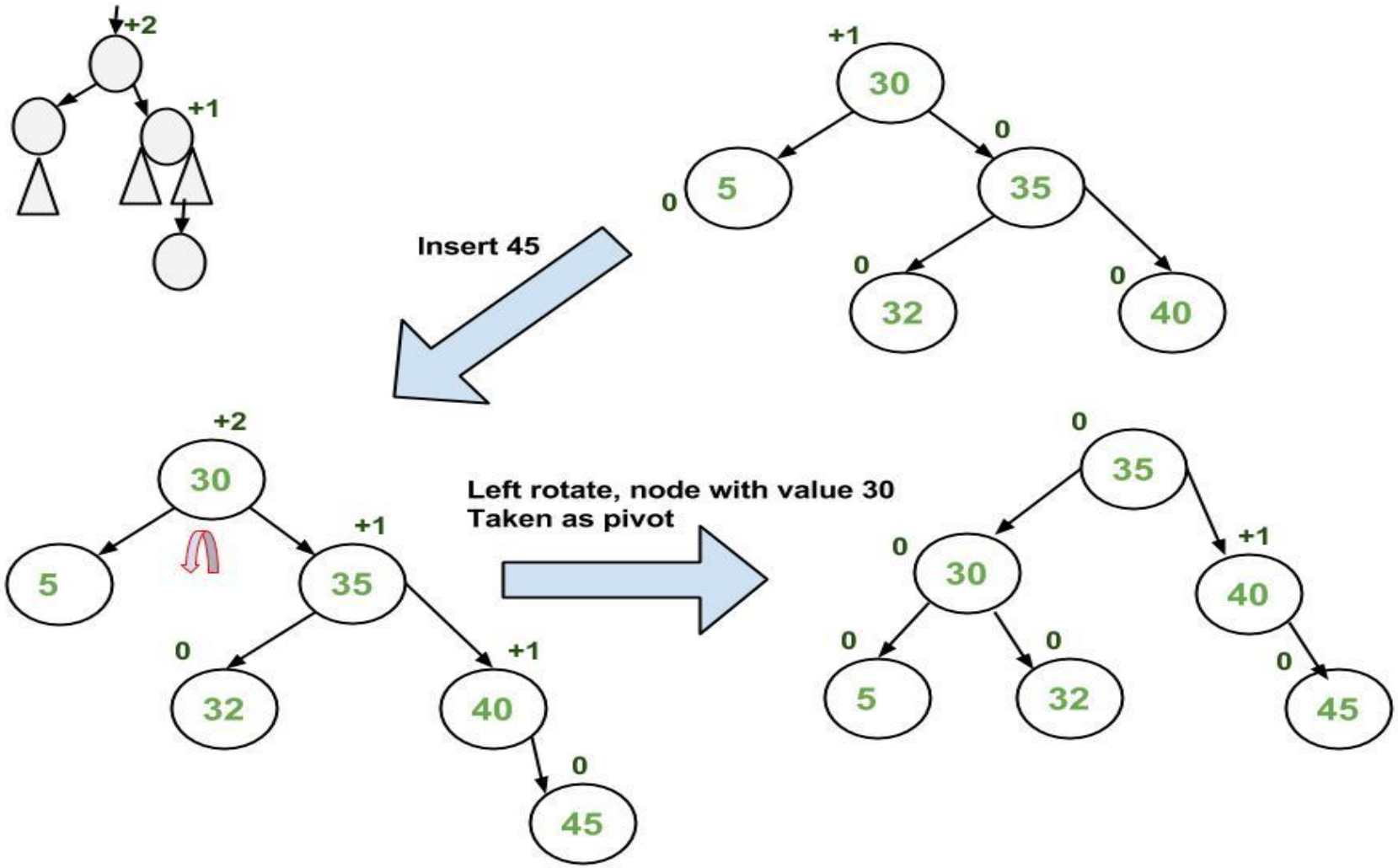
Ví dụ thêm nút vào cây NPTK CB: LL



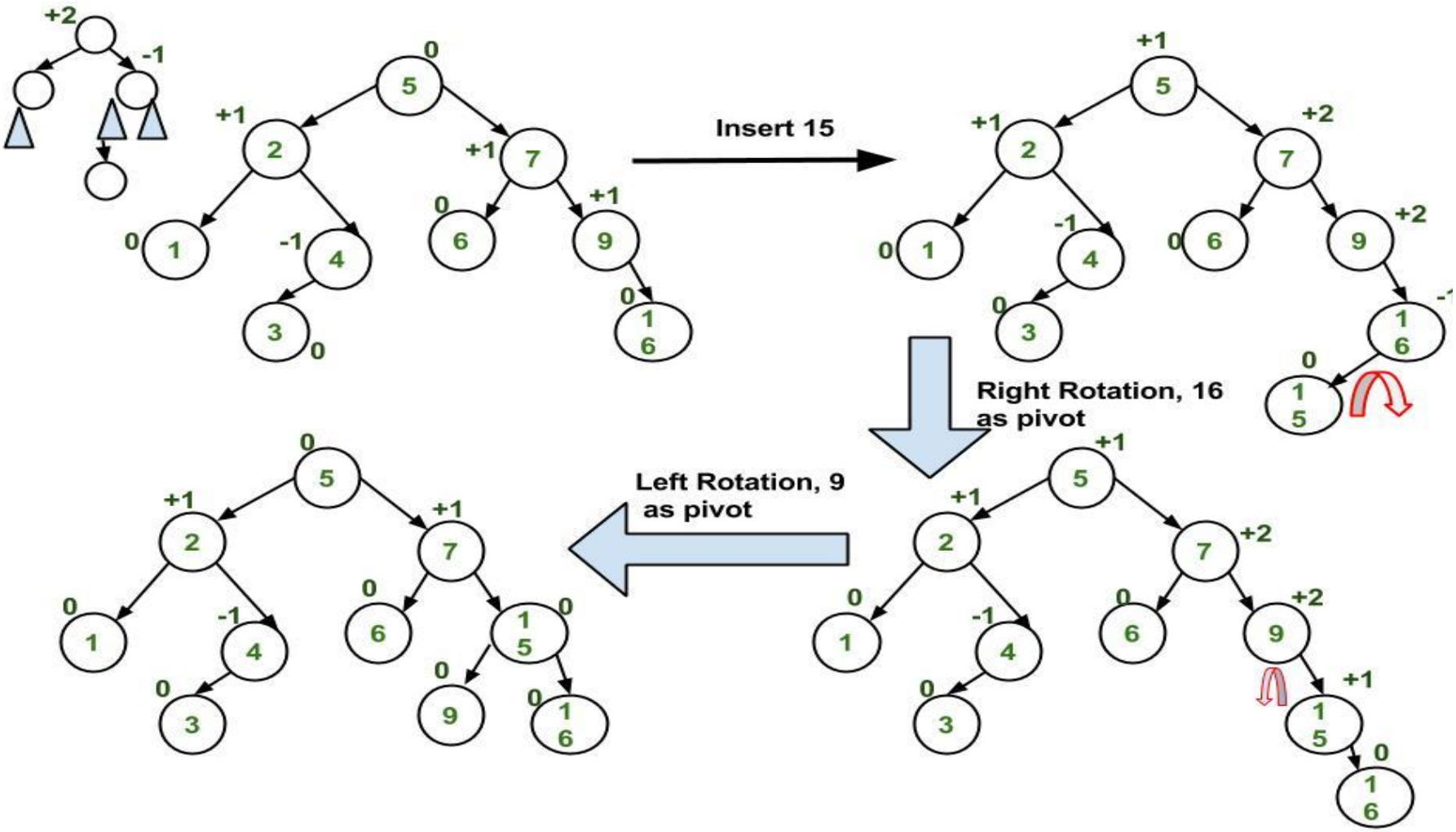
Ví dụ thêm nút vào cây NPTK CB: LR



Ví dụ thêm nút vào cây NPTK CB: RR



Ví dụ thêm nút vào cây NPTK CB: RL



Hàm thêm nút vào cây NPTK CB

```
NODE* insert(TREE &t, int val)
{
        NODE* p=create_node(val);
        /* 1.  Perform the normal BST insertion */
        if (t == NULL)
                return   p;
        if (val < t->key)
                t->pLeft = insert(t->pLeft, val);
        else if (val >t->key)
                t->pRight = insert(t->pRight, val);
        else // Equal keys are not allowed in BST
                return t;
```

Thêm nút vào cây NPTK CB

```
/* 2. Update height of this ancestor node */
        t->height = max(height_tree(t->pLeft), height_tree(t->pRight)) + 1;
        /* 3. Get the balance factor of this ancestor node to check whether this node became unbalanced */
        int bal = get_balance(t);
        // Left Left Case
        if (bal > 1 && val < t->pLeft->key)
                return rotation_right(t);
        // Right Right Case
        if (bal <-1 && val > t->pRight->key)
                return rotation_left(t);
```

Thêm nút vào cây NPTK CB

```
// Left Right Case
if (bal>1 && val > t->pLeft->key)
{
        t->pLeft = rotation_left(t->pLeft);
        return rotation_right(t);
}
// Right Left Case
if (bal < -1 && val < t->pRight->key)
{
        t->pRight = rotation_right(t->pRight);
        return rotation_left(t);
}
return t;
}
```

Một số hàm hỗ trợ

```
int max(int a, int b)
{
        return (a > b) ? a : b;
}
int height_tree(TREE t)
{
        if (t == NULL)
                return 0;
        return t->height;
}
int get_balance(TREE &t)
{
        if (t == NULL)
                return 0;
        return height_tree(t->pLeft) - height_tree(t->pRight);
}
```

Một số hàm hỗ trợ

```
//Tìm phần tử thế mạng
NODE* min_value_node(NODE* p)
{
        NODE* curr = p;
        while (curr->pLeft != NULL)
                curr = curr->pLeft;
        return curr;
}
```

Hủy nút trên cây NPTK CB

- Hủy bình thường như trường hợp cây NPTK.
- Nếu cây giảm chiều cao:
  - Lần ngược về gốc để phát hiện nút bị mất cân bằng.
  - Tiến hành cân bằng lại nút đó bằng thao tác cân bằng thích hợp.
  - Tiếp tục lần ngược lên nút cha…
- Việc cân bằng lại có thể lan truyền lên tận gốc.

Hủy nút trên cây NPTK CB

Let w be the node to be deleted.

1) Perform standard BST delete for w.

2) Starting from w, travel up and find the first unbalanced node. Let z be the first unbalanced node, y be the larger height child of z, and x be the larger height child of y. Note that the definitions of x and y are different from insertion here.

Hủy nút trên cây NPTK CB

3) Re-balance the tree by performing appropriate rotations on the subtree rooted with z. There can be 4 possible cases that needs to be handled as x, y and z can be arranged in 4 ways. Following are the possible 4 arrangements:

a) y is left child of z and x is left child of y (Left Left Case).

b) y is left child of z and x is right child of y (Left Right Case).

c) y is right child of z and x is right child of y (Right Right Case).

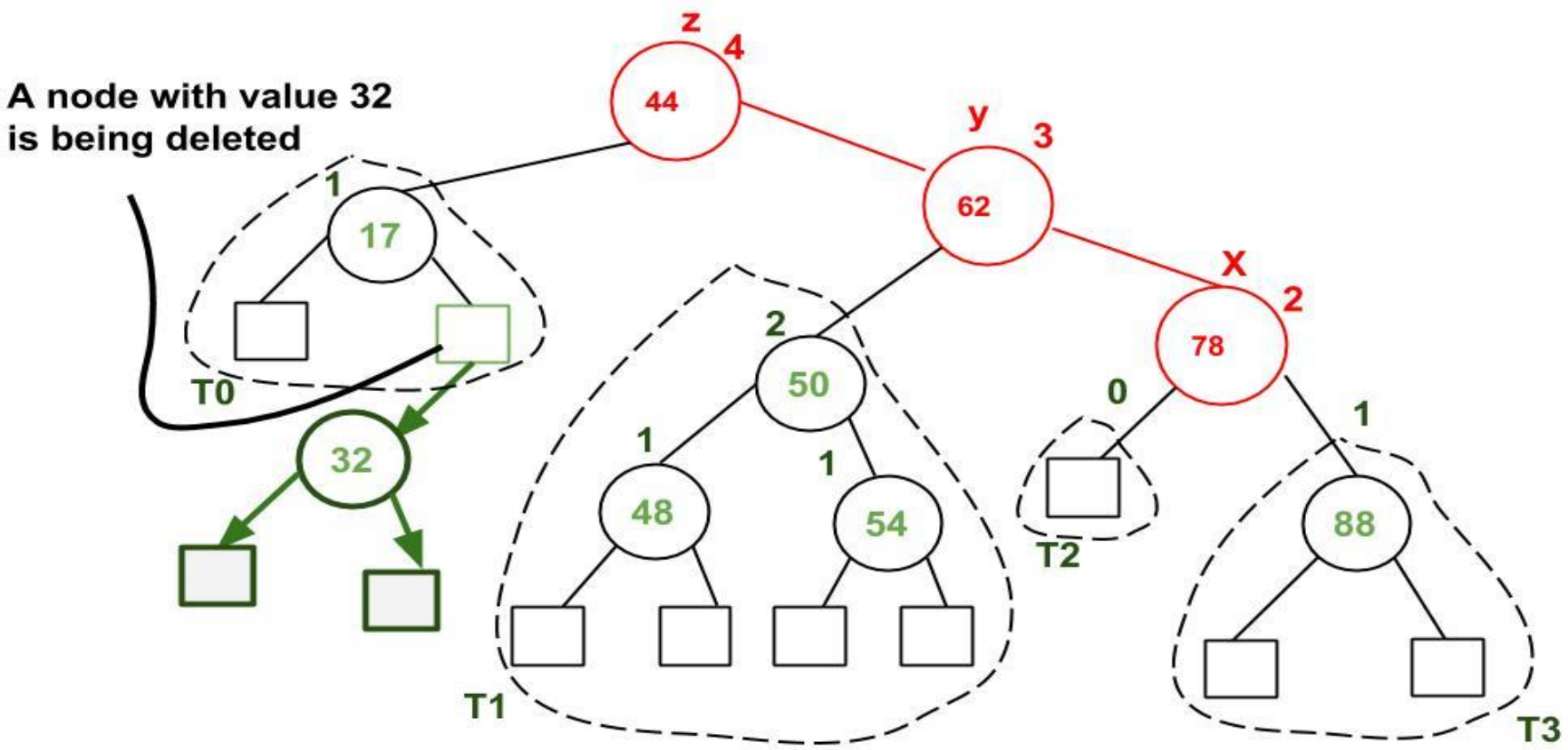
d) y is right child of z and x is left child of y (Right Left Case).

Hủy nút trên cây NPTK CB

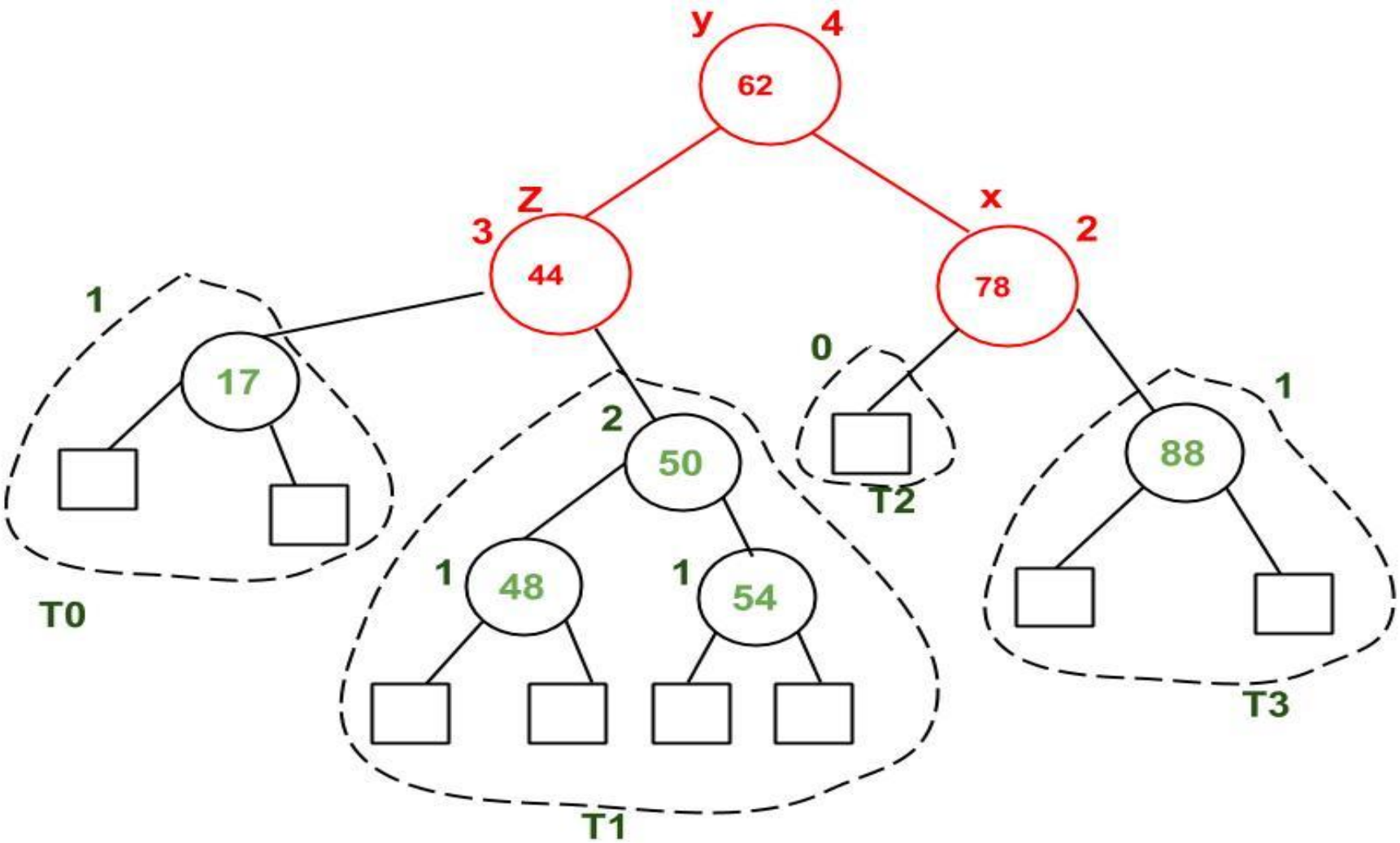
- Like insertion, following are the operations to be performed in above mentioned 4 cases.
- Note that, unlike insertion, fixing the node z won't fix the complete AVL tree. After fixing z, we may have to fix ancestors of z as well.

Ví dụ hủy nút trên cây NPTK CB

Example of deletion from an AVL Tree:



Ví dụ hủy nút trên cây NPTK CB



Hàm hủy nút trên cây NPTK CB

```
NODE* deletion(TREE &t, int val)
{
        // STEP 1: PERFORM STANDARD BST DELETE
        if (t == NULL)
                return t;
        if (val < t->key)
                t->pLeft = deletion(t->pLeft, val);
        else if (val > t->key)
                t->pRight = deletion(t->pRight, val);
        else // Equal keys are not allowed in BST
        {
                // node with only one child or no child
                if (t->pLeft == NULL || t->pRight == NULL)
                {
                        NODE* temp = t->pLeft ? t->pLeft : t->pRight;
                        // No child case
```

Hàm hủy nút trên cây NPTK CB

```
if (temp == NULL)
{
        temp = t;
        t = NULL;
}
else
        // One child case
        *t = *temp; // Copy the contents of the non-empty child
delete temp;
}
```

Hàm hủy nút trên cây NPTK CB

```
        else
        {
                NODE* temp = min_value_node(t->pRight);
                t->key = temp->key;
                t->pRight = deletion(t->pRight, temp->key);
        }
    }
    if (t == NULL)
        return t;
/* 2. Update height of this ancestor node */
/* 3. Get the balance factor of this ancestor node to check whether this node became unbalanced */
// Left Left Case          // Right Right Case
// Left Right Case         // Right Left Case
```

Bài tập chương 5

Bài 1: Vẽ hình cây nhị phân tìm kiếm tạo ra từ cây rỗng bằng cách lần lượt thêm vào các khoá là các số nguyên: 54, 31, 43, 29, 65, 10, 20, 36, 78, 59.

Bài 2: Cho biết kết quả khi duyệt trước, duyệt giữa và duyệt sau (từ trái qua phải) khi duyệt cây nhị phân ở bài 1

Bài 3: Vẽ lại cây nhị phân bài 1 sau khi xóa phần tử 31 khỏi cây.

Bài 4: Vẽ cây nhị phân tìm kiếm cân bằng từ các số nguyên ở bài 1.

Bài 5: Viết chương trình nhập vào 1 cây NPTK. Duyệt qua cây theo các cách: duyệt trước, duyệt giữa và duyệt sau.